数据与算法

CH04 栈与队列

北京科技大学
计算机与通信工程学院
杜　冰/皇甫伟

习题 1：设计一个存储整数的线性表，根据标准输入对线性表进行操作。

输入：若干行；
输出：按要求进行输出。

输入是若干行，每行是一个命令，包括：
创建线性表，命令为"C m"，其中 m 是非负整数，表示创建一个 m 长度的线性表，并依次填充从 0 到 m-1 的整数。若 m 为 0 则创建空表。测试用例保证 C 是第一个命令，且一定是合法的。

插入整数，命令为"I x y"，其中 x 和 y 均为整数，x 非负，为插入位置，y 是待插入的值。如"I 3 7"表明在第 3 个位置（自 0 开始编号）插入整数 7。"I 0 5"表示在首部插入整数 5。如果 x 不合法，应输出错误信息"X"并停止程序。

删除整数，命令为"D x"，其中 x 为整数，表示删除的位置。如"D 3"表明删去第 3 个位置（自 0 开始编号）的元素。"D 0"表示删去首部的元素。如果 x 不合法，应输出错误信息"X“(井号)并停止程序。

批量删除整数，命令为"E x y"，其中 x 和 y 均为整数，表示删除的区间。如"E 3 5"表明删去第 3 个位置到第 5 个位置（自 0 开始编号）的元素，含第 3 和第 5。"D 0 1"表示删去首部和随后的元素。如果 x 和 y 不合法，应输出错误信息"X“(井号)并停止程序。

反转线性表：命令为"R"，表示将现有的线性表逆序重排。

输出线性表，命令为"P"，表示自首部到尾部依次输出线性表中的各元素，以空格分开，程序退出。对空表输出"EMPTY"，程序退出。

注意执行上述命令序列的过程中，程序只会有 1 行输出（检测到错误或输出线性表）。

测试用例 1：

输入：

C 3 （注：线性表 L=0 1 2）

I 1 5 （注：L=0 5 1 2)

D 2（注：L=0 5 2)

P

输出：

0 5 2

测试用例 2：

输入：

C 3 （注：线性表 L=0 1 2）

D 3 （注：非法删除，忽略后续的输入，程序退出）

输出：

X

本节内容

- 基本概念
- 栈与递归
- 队列

基本概念

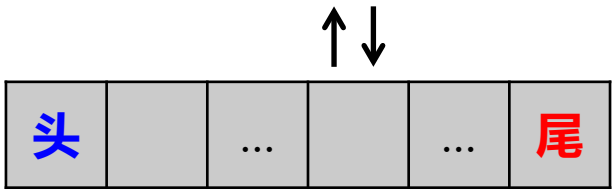
- **比线性表更简单？任意位置数据元素的操作是否总是必要？**

- **很多时候，仅需要对线性表的头/尾数据操作：**
  - 栈(stack)、队列(queue)、双端队列(deque)

基本概念

- **栈、队列、双端队列均是线性表的特殊情况（访问受限的线性表）**

一般线性表
'随进随出(RIRO)'



Insert()
Delete()

栈
'后进先出(LIFO)'

Push()
Pop()

队列
'先进先出(FIFO)'

EnQueue()
DeQueue()

双端队列
'先后进出?'

EnQueue_Front()
Dequeue_Front()
EnQueue_Back()
DeQueue_Back()

基本概念

- 栈：暂存问题稍后处理（e.g. 子函数调用）
- 队列：待处理的事件排队（e.g. 网络包路由）
- 双端队列：更灵活的需求（e.g.插销DeQueue/EnQueue操作）

本节内容

- 基本概念
- 栈与递归
- 队列

栈与递归

- 栈：限制数据插入、删除操作只能在一端进行的特殊线性表
- 允许插入、删除的一端称为栈顶(top)，另一端称为栈底(bottom)
- 在栈顶插入元素称为入栈(push)，删除元素称为出栈(pop)
- 栈中元素的个数称为栈的长度

栈与递归

- **栈的ADT**

```
ADT Stack{
    数据对象: D={a_i | a_i∈ElemSet, i=0,1,…,n-1}
    数据关系: R={<a_i, a_i+1> | a_i∈D, i=0,1,…,n-2}
              a_0为栈底, a_n-1为栈顶.
    基本操作:

    InitStack(&S)
        操作结果: 构造一个空栈.

    DestroyStack(&S)
        初始条件: 栈S已存在.
        操作结果: 销毁栈S.
```

栈与递归

……

StackEmpty(S)

初始条件：栈S已存在.

操作结果：若S空，返回1，否则返回0.

StackLength(S)

初始条件：栈S已存在.

操作结果：返回S的长度n.

GetTop(S,&e)

初始条件：栈S已存在且非空.

操作结果：用e返回S的栈顶元素值.

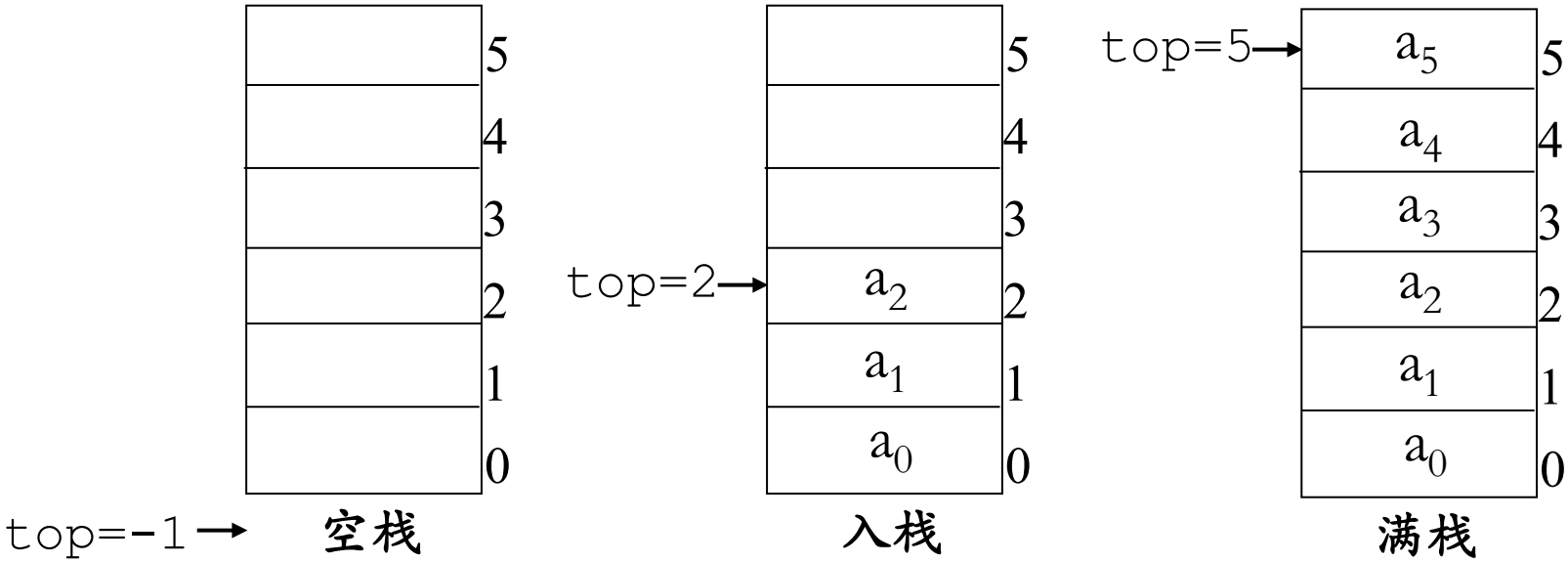
……

栈与递归

```
……
    Push(&S,e)
        初始条件：栈S已存在.
        操作结果：插入e为新的栈顶元素.

    Pop(S, &e)
        初始条件：栈S已存在且非空.
        操作结果：删除栈顶元素，并用e返回其值.

    Clear(&S)
        初始条件：栈S已存在.
        操作结果：清空栈S，清空后栈长度为0.

} ADT Stack
```

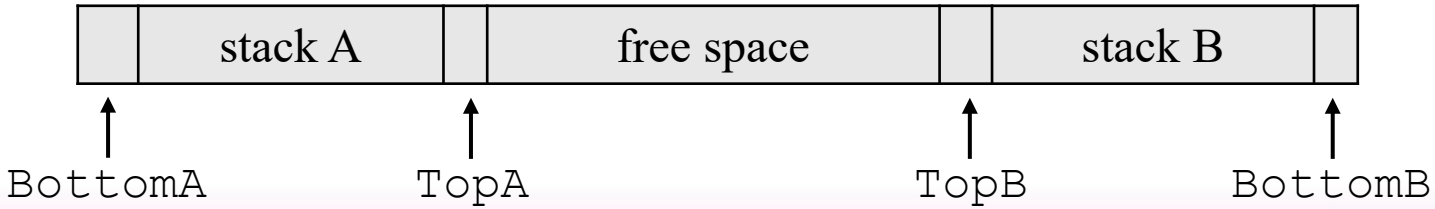
栈与递归

- **顺序栈 - 栈的顺序存储实现**
  - 用一维数组stack[MaxSize]存储栈



栈与递归

- **顺序栈存储空间的处理**
  - 静态分配空间
    - ElemType stack[MaxSize]
  - 初始化时动态分配
    - ElemType* stack = new ElemType[MaxSize]
  - 栈溢出时扩容
    - StackInc = new ElemType[MaxSize+IncSize]
  - 双顺序栈共享存储空间



顺序栈的C++实现

```
template <class ElemType>
class SeqStack{
    private:
        ElemType* stack; int MaxSize; int top;
    public:
        SeqStack(int m)
          { MaxSize=m; stack=new ElemType[m]; top=-1; }
        ~SeqStack() { delete[] stack; }
        void ClearStack() { top=-1; }
        bool StackEmpty() { return top == -1; }
        bool StackFull()  { return top == MaxSize-1; }
        int StackLength() { return top+1; }
        bool Push(ElemType e)
           { if(StackFull()) return false;
             stack[++top]=e; return true;}
        bool Pop(ElemType &e)
           { if(StackEmpty()) return false;
             e = stack[top--]; return true;}
        bool GetTop(ElemType &e)
           { if(StackEmpty()) return false;
             e = stack[top]; return true;}
};
```

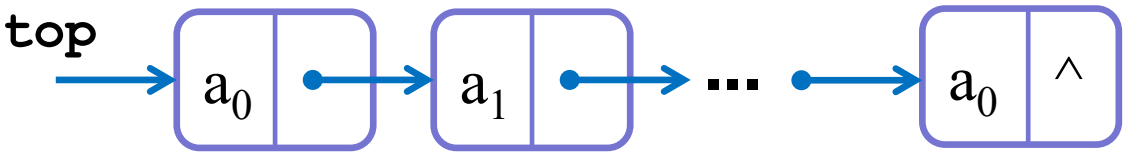
注意一元操作符的顺序

栈与递归

- **顺序栈的特点**
  - 顺序栈实现简单
  - 顺序栈的入栈和出栈，时间复杂度均O(1)
  - 对简单数据类型，构造和销毁栈也是O(1)
  - 栈空间最大长度在构造时确定，空间利用率可能不理想，且需要判断溢出
  - 可以采用溢出时增容，或者共享栈的方式提高空间利用率

栈与递归

- **链式栈 - 栈的链式存储分配实现**
  - 用链表的方式存储栈




```
template <class ElemType>
class Node{
    public:
        ElemType data;
        Node<ElemType> * next;
        Node(ElemType e, Node* t)
                {data=e; next=t;}
}
```

栈与递归

链式栈的C++实现

```
template <class ElemType>
class LkStack{
    private:
        Node* top; int len;
    public:
        SeqStack() {top=NULL; len=0;}
        ~SeqStack() { ClearStack(); }
        void ClearStack();
        bool StackEmpty() { return len==0; }
        int StackLength() { return len; }
        Push(ElemType e);
        bool Pop(ElemType &e);
        bool GetTop(ElemType &e)
}
```

```
template <class ElemType>
void LkStack<ElemType>::ClearStack(){
    ElemType e; while(pop(e));
}

template <class ElemType>
void LkStack<ElemType>::Push(ElemType e){
    Node* s=new Node(e, top); top = s; len++; }

template <class ElemType>
bool LkStack<ElemType>::Pop(ElemType &e){
    if(StackEmpty()) return false;
    e=top->data; Node* q=top; top=top->next;
    delete q; len--; return true;  }

template <class ElemType>
bool LkStack<ElemType>::GetTop(ElemType &e){
    if(StackEmpty()) return false;
    e=top->data; return true; }
```

栈与递归

- **链式栈的特点**
  - 链式栈的入栈和出栈，时间复杂度均O(1)
  - 构造和销毁栈时间复杂度为O(n)
  - 栈长度动态变化，可以有效利用空间
  - 一般无需判断溢出

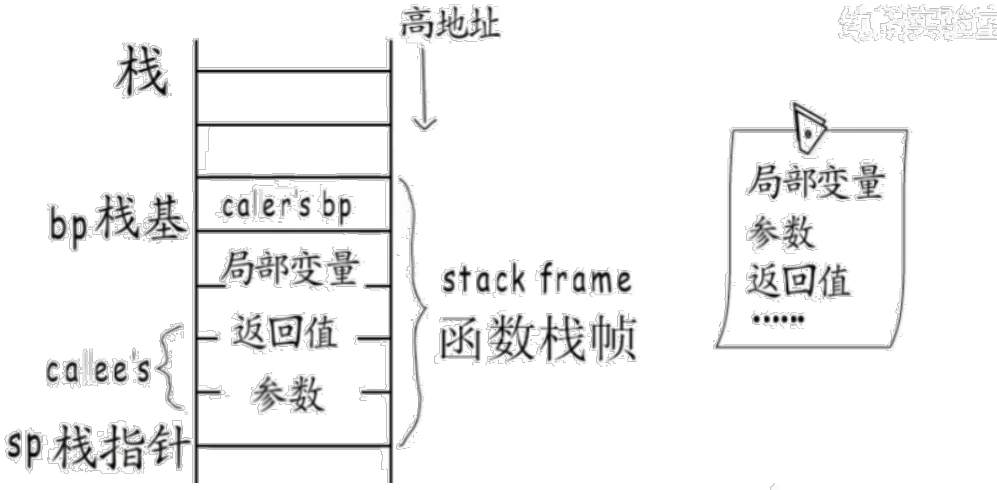
栈与递归

- **栈的应用性**
  - 栈应用广泛，使用频率非常高（函数调用）
  - 常适用于将问题暂存转而解决子问题：
    - 显示应用：表达式求值，floor planning，图搜索（深度优先搜索，Depth First Search）等
    - 隐式应用：函数调用，递归
  - 在其他领域也有应用或借鉴其思想：
    - 系统结构：存储器管理
    - 网络：协议栈，栈溢出（stack smashing）

函数调用例子

- swap函数栈帧

1. 分配局部变量的空间，然后把局部变量存进去
2. 函数返回值
3. 分配被调用函数的参数，从右至左分配。先入栈第二个参数，再入栈第一个参数。传参是值拷贝，所以把两个参数的值压入栈
4. call指令存入的返回地址。也就是fmt.Println这一行代码所对应的指令



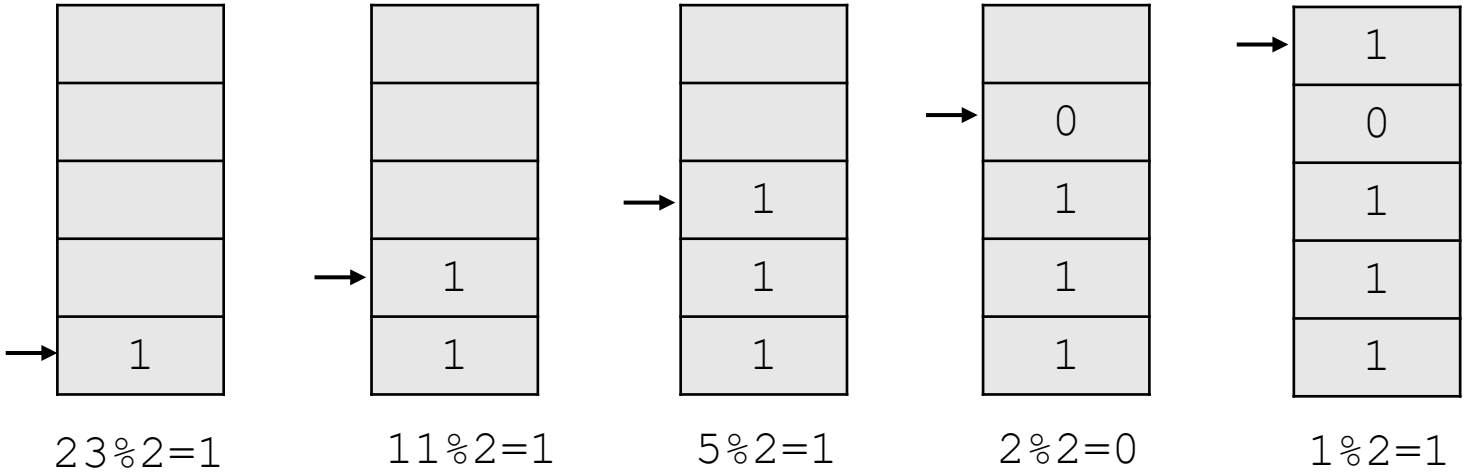

```
func swap(a,b int) {
    a,b = b,a
}

func main() {
    a,b := 1,2
    swap(a,b)
    fmt.Println(a,b)  //1,2
}
```

栈与递归

- 十进制数到二进制数的转换
- 表达式求值
- 过程的嵌套调用

栈与递归

- **十进制数到二进制数的转换**
  - 23 -> 10111; 2587964 -> ?
  - N = (N div 2)×2+N mod 2
  - 对2取余依次入栈；依次出栈即二进制表示
  - 可推广到其他进制



栈与递归

```
void dec2bin(unsigned int N)
    LkStack<int> S;
    while(N>0){
        S.push(N%2);                    //余数入栈
        N = floor(N/2);
    }
    if(S.StackEmpty()) cout<<0<<endl; //输入为0
    else{
        int e;
        while(!S.StackEmpty()){
            S.pop(e);                   //依次出栈
            cout<<e;
        }
        cout<<endl;
    }
}
```

dec2bin(2587964): '1001110111110100111100'

栈与递归

- **算术表达式求值**
  - 中缀表达式：$'9+6\times(4-1);'$，以 ‘;’ 结束；
  - 两类符号：操作数(operand)，运算符(operator);
  - 运算符之间存在优先级的问题.

运算符	×	/	+/-	(	)	;
优先级	2	2	3	1	4	0

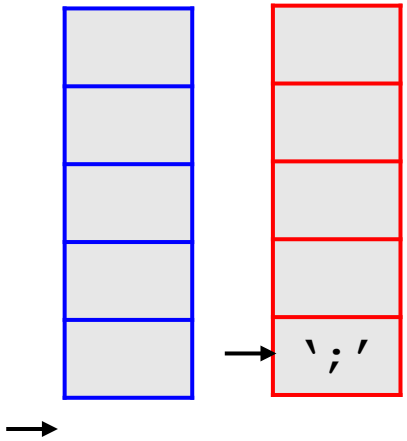
栈与递归

- **算术表达式求值的栈实现**
  - 建立两个栈：Sd存放操作数，Sr存放运算符；
  - 自左向右扫描表达式，根据当前符号W进行操作：
    - 结束符‘;’入栈：Sr.Push(‘;’)
    - 如果W是操作数则入栈：Sd.Push(W)
    - 若W为‘(’则入栈： Sr.Push(‘(’)
    - 若W为‘)’且Sr栈顶元素为‘(’则出栈：Sr.Pop(e)
      - 若W优先级高于Sr栈顶元素则入栈：Sr.Push(W)
      - 若W优先级不高于Sr栈顶元素则执行运算操作：
      - Sd.Pop(x1); Sd.Pop(x2); Sr.Pop(@); Sd.Push(x2@X1)
      - 若W为‘;’且Sr栈顶元素为‘;’则运算终止
      - Sd栈顶即为运算结果

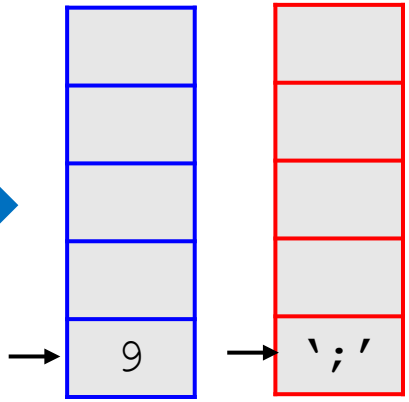
栈与递归

Sd.Push(x2@X1)

$9+6\times(4-1);$

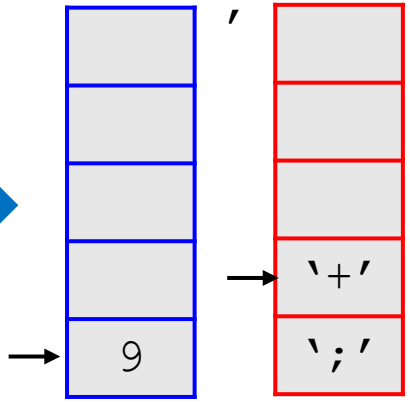


$9+6\times(4-1);$

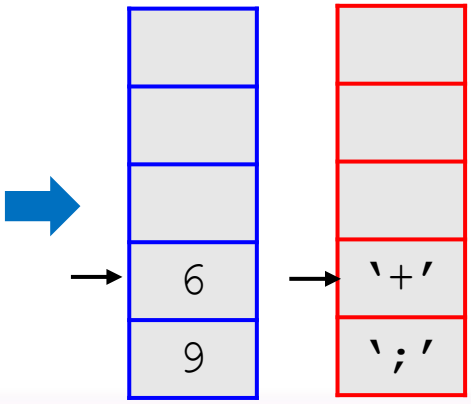


$9+6\times(4-1);$

'+' > ';'

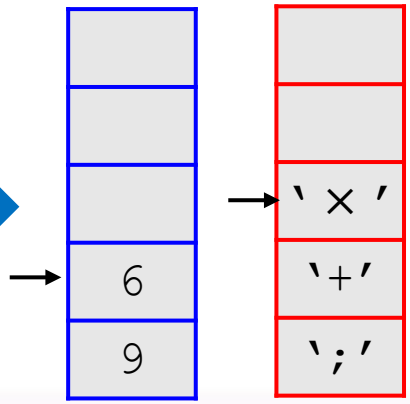


$9+6\times(4-1);$

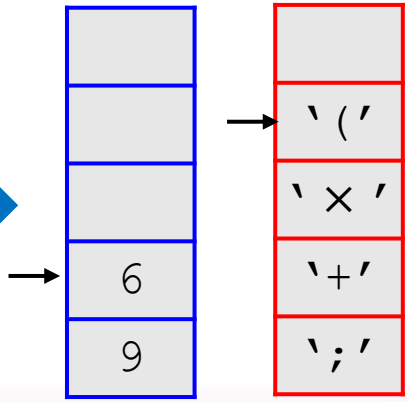


$9+6\times(4-1);$

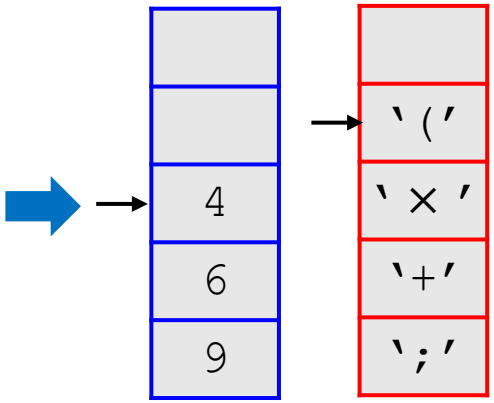
'×' > '+'



$9+6\times(4-1);$

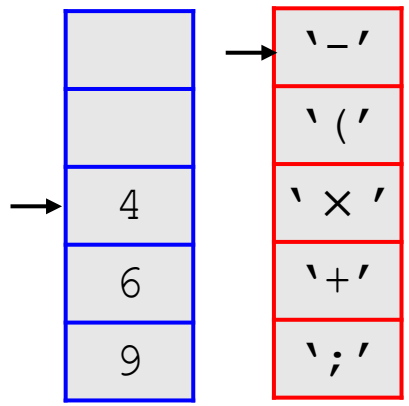


$9+6\times(4-1)$

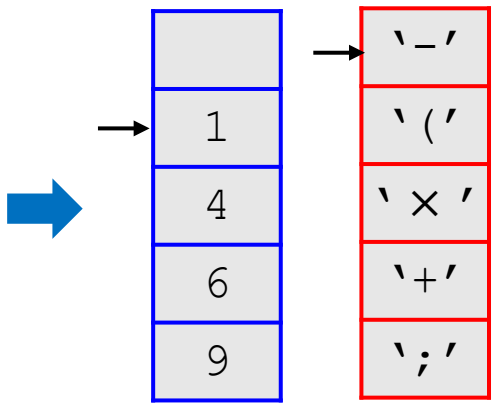


$9+6\times(4-1)$

'–' > '('

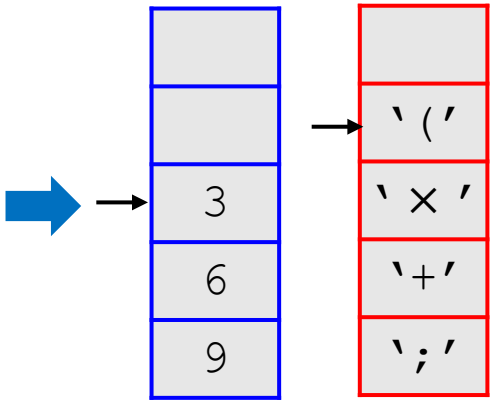


$9+6\times(4-1)$



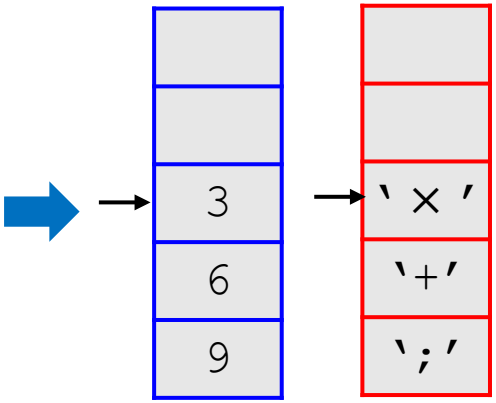
$9+6\times(4-1)$

4−1=3 ')' < '–'



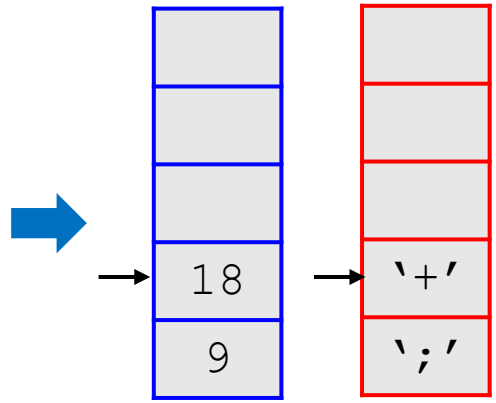
$9+6\times(4-1)$

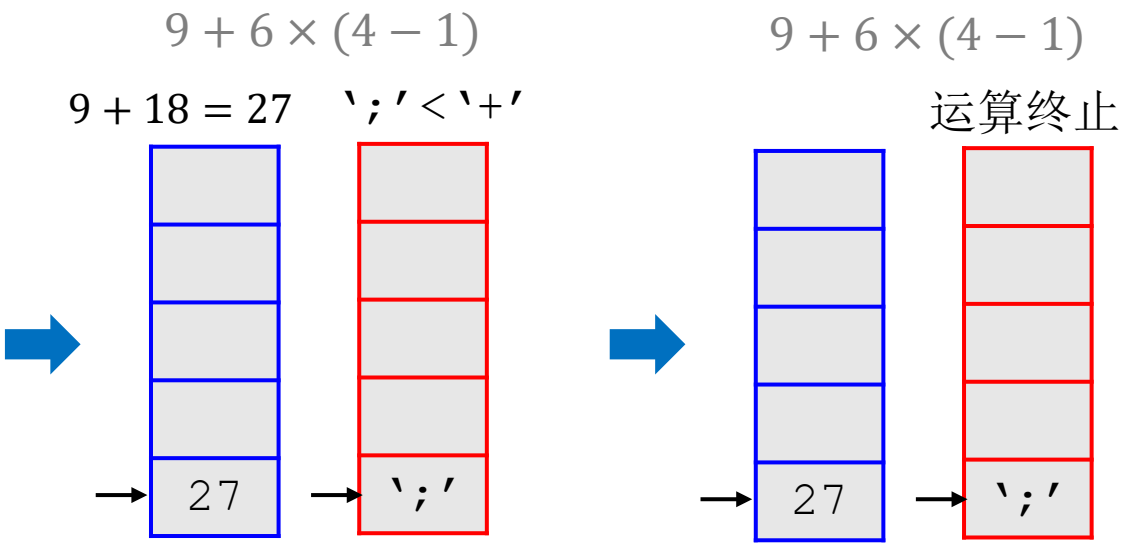
'(' 出栈



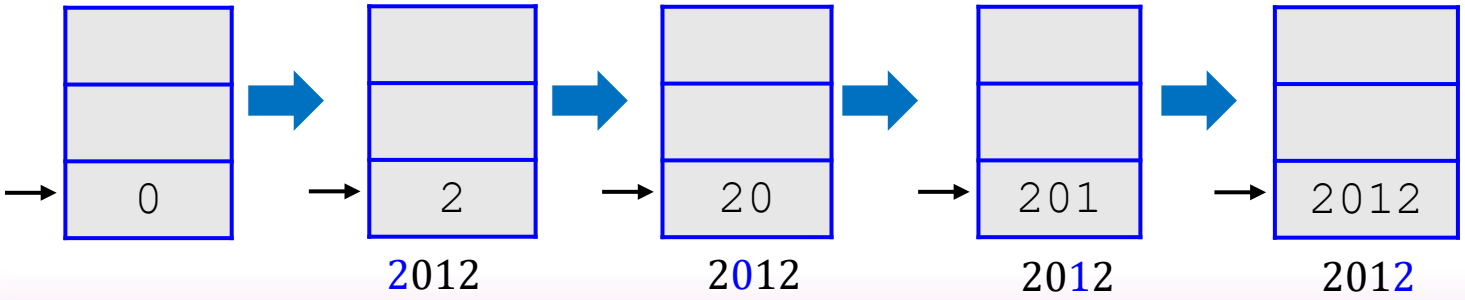
$9+6\times(4-1)$

$6\times3=18$ '+' < '×'





- **注意事项**
  - 后出栈的在前作为被运算数 Sd.Push(x2@X1)
  - 如运算数有多个字符：



栈与递归

- **后缀表达式postfix（逆波兰表达式 Reverse Polish）**
  - 1929年由波兰数学家Lukasiewics提出；
  - 运算符置于参与运算的操作数之后；
  - 没有括号，也不存在运算符优先级的问题；
  - 用于表达式求值非常方便.

中缀表达式	后缀表达式
$A+B$	$AB+$
$A+B\times C$	$ABC\times+$
$(A+B)\times C$	$AB+C\times$
$A+B\times(C-D)/E$	$ABCD-\times E/+$

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出直到遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

–
(
(
(
×

5	25	8							

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出直到遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

(
(
×

5	25	8	−						

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

×
(
(
×

5	25	8	–						

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

(
×
(
(
×

5	25	8	−	48					

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

/
(
×
(
(
×

5	25	8	–	48	2				

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

×
(
(
×

5	25	8	–	48	2	/			

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

(
×

5	25	8	−	48	2	/	×		

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

+
(
×

5	25	8	−	48	2	/	×	7	

栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

5	25	8	−	48	2	/	×	7	+

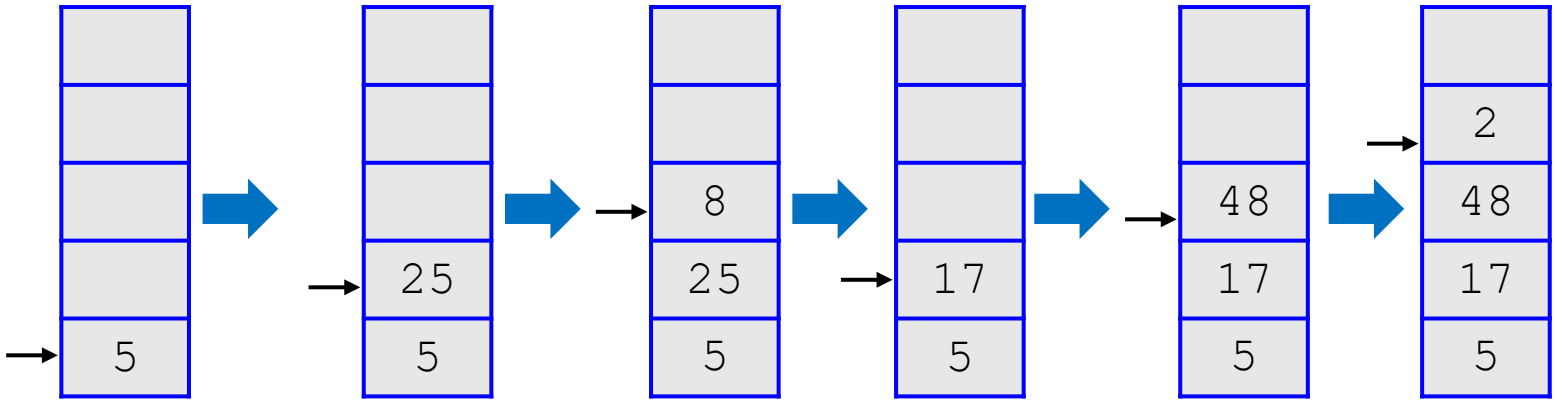
栈与递归

- 中缀表达式 $5 \times (((25-8) \times (48/2)) + 7)$
- 后缀表达式运算：
  - 操作数入栈，但运算符并不立即输出，而存入栈中
  - 如果遇到左括号则入栈，遇到右括号就将栈元素弹出，弹出符号输出知道遇到对应的左括号
  - 如果遇到任何其他符号（+，*，-，/），从栈中弹出栈元素直到发现优先级更低的元素为止
  - 运算符则两个操作数出栈运算，结果进栈

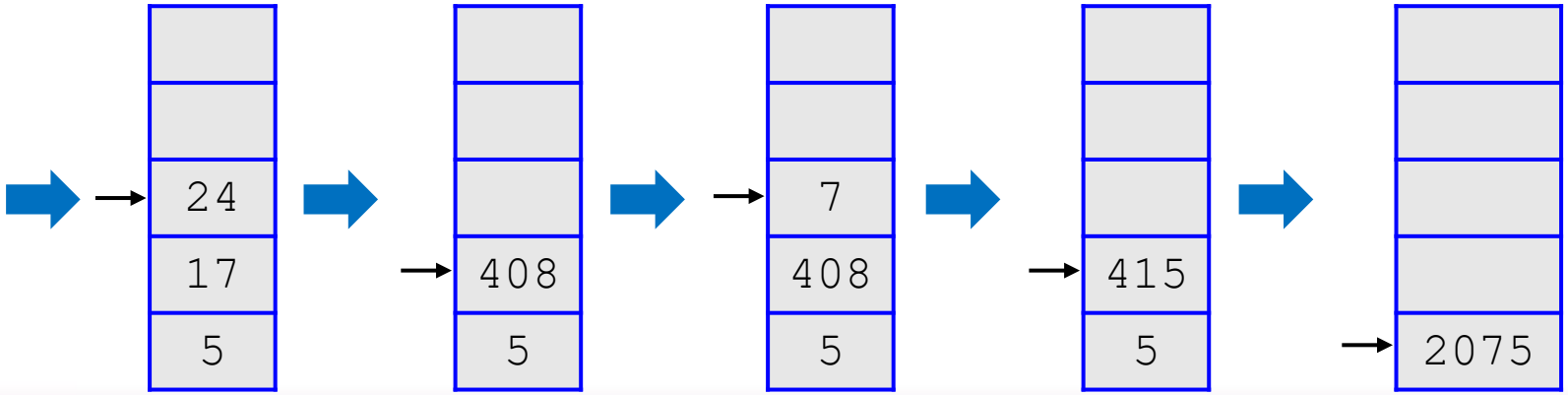
5	25	8	−	48	2	/	×	7	+	×

$5\ 25\ 8 - 48\ 2 / \times\ 7 + \times$

$25 - 8 = 17$



$48/2 = 24$ $17 \times 24 = 408$ $7 + 408 = 415$ $415 \times 5 = 2075$



```
char *a="5 25 8-48 2/*7+*"
...
for(int i=0;i<strlen(a);i++){
   if(a[i]==' ');            //空格为间隔符
   if(a[i]=='+')
    {s.pop(e1);s.pop(e2);s.push(e2+e1);}
   if(a[i]=='-')
    {s.pop(e1);s.pop(e2);s.push(e2-e1);}
   if(a[i]=='*')
    {s.pop(e1);s.pop(e2);s.push(e2*e1);}
   if(a[i]=='/')
    {s.pop(e1);s.pop(e2);s.push(e2/e1);}
...
```

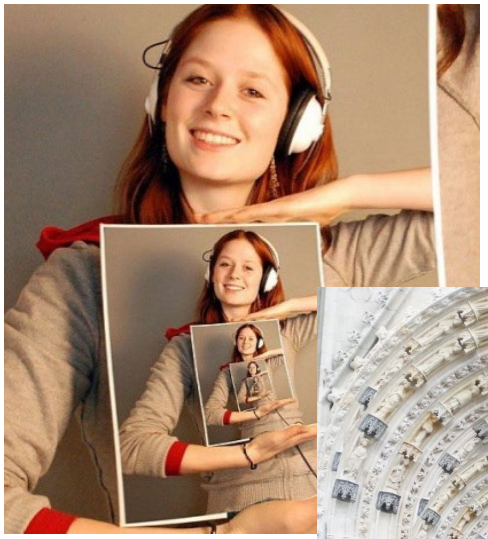
栈与递归

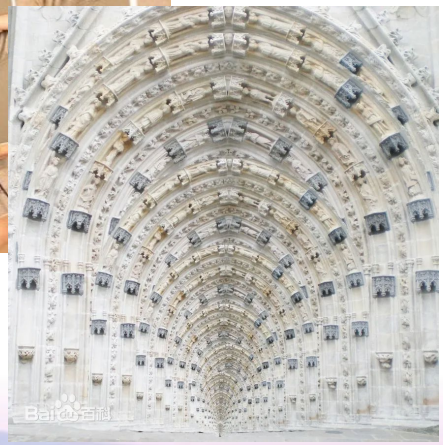
```
...
    if(a[i]>='0'&&a[i]<='9'){
        s.push(0);      //处理多位整数
        while(a[i]>='0'&&a[i]<='9'){
            s.Pop(e);
            s.Push(e*10+(a[i]-'0'));
            i++;
        }
        i--;
    }
}
...
```

- **小数点的处理?**
- **除零错?**
- **一元操作符（正负号）的处理?**
- **非法表达式?**

栈与递归

- **递归(recursion) 对象的递归:对象部分包含自身，或者用自己给自己定义**
  - 过程的递归：一个过程直接或者间接地调用自己.
  - 递归调用
    - 支持：C/C++, PASCAL, Java, ...
    - 不直接支持: BASIC，...;





```
template <class ElemType>
class Node{
    public:
        ElemType data;
        Node<ElemType> * next;
}
```

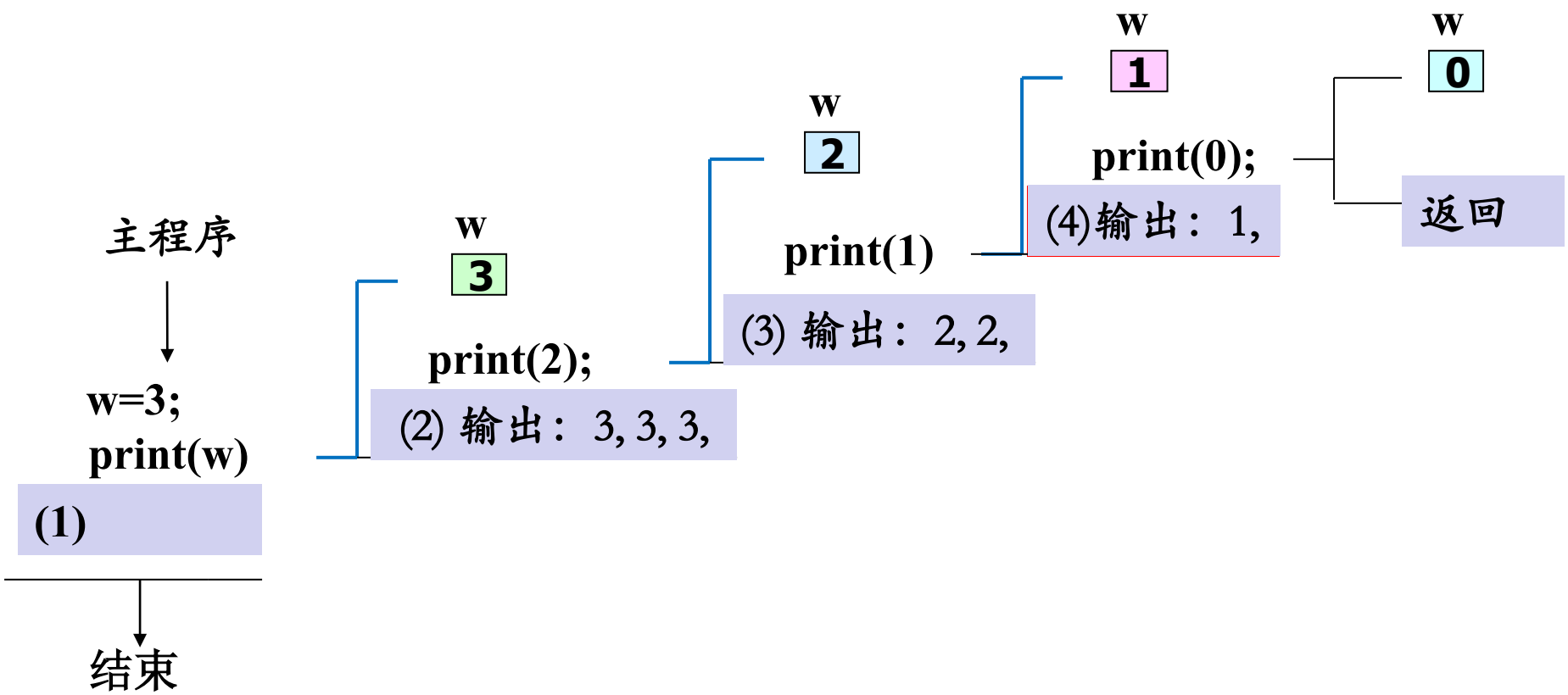
```
void print(unsigned int  w){
    if ( w!=0)
    {  print(w-1);
       for(int i=1; i<=w; i++)
           printf("%3d,",w);
        printf("/n");
     }
}
```

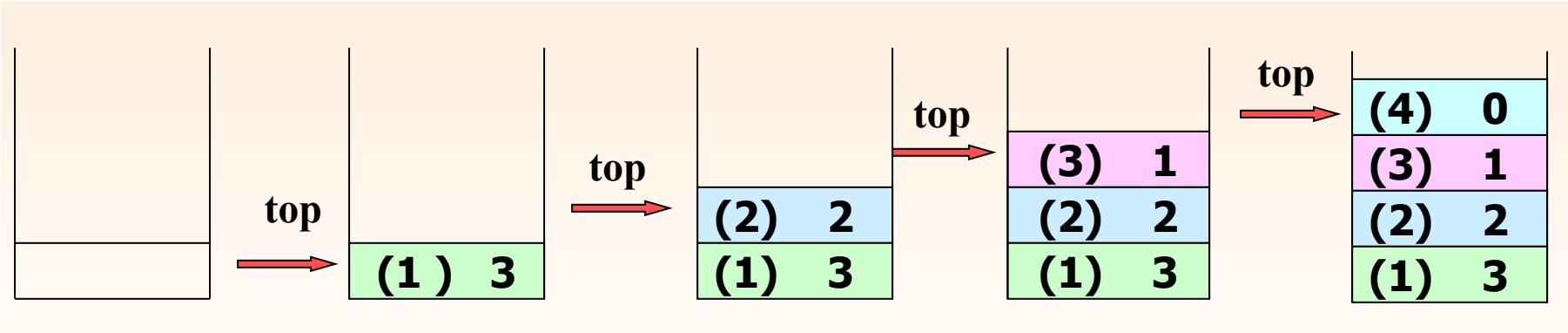
栈与递归

```
void print(unsigned int  w){
    if ( w!=0)
    {  print(w-1);
       for(int i=1; i<=w; i++)
           printf("%3d,",w);
       printf("/n");
     }
}
```

print(3);

运行结果:
1,
2, 2,
3, 3, 3,


主程序
w=3;
print(w)
(1)
结束
w
3
print(2);
(2) 输出： 3, 3, 3,
w
2
print(1)
(3) 输出： 2, 2,
w
1
print(0);
(4)输出： 1,
w
0
返回


top
(1) 3
top
(2) 2
(1) 3
top
(3) 1
(2) 2
(1) 3
top
(4) 0
(3) 1
(2) 2
(1) 3

栈与递归

- **递归的适用情况**
  - 问题的定义本身就是递归的；
  - 问题的数据结果是递归的；
  - 问题的处理过程可以归结成递归形式.

> "The power of recursion evidently lies in the possibility of defining an infinite set of objects by a finite statement. In the same manner, an infinite number of computations can be described by a finite recursive program, even if this program contains no explicit repetitions."
>
> [Wirth & Niklaus, 1976]

栈与递归

- 阶乘求值 (factorial)
- 阿克曼函数 (Ackermann)
- 斐波那契数列 (Fibonacci)
- 最大公约数 (greatest common divisor)
- 互递归 (mutual recursion)
- 汉诺塔问题 (Tower of Hanoi)

栈与递归

- **阶乘求值** $n! = 1 \times 2 \times 3 \times \cdots \times n$；
- **递归表达** $n! = \begin{cases} 1 & n = 0 \\ n \times (n-1)! & n > 0 \end{cases}$；
- **递归实现**

```
int factorial(unsigned int  n){
    if(n==0) return 1;
    else return n*factorial(n-1);}
```

$$
\begin{aligned}
f(3) &= 3 \times f(2) \\
&= 3 \times (2 \times f(1)) \\
&= 3 \times (2 \times (1 \times f(0))) \\
&= 3 \times (2 \times 1) \\
&= 3 \times 2 \\
&= 6
\end{aligned}
$$

$$
\begin{aligned}
3! &= 3 \times 2! \\
&= 3 \times (2 \times 1!) \\
&= 3 \times (2 \times (1 \times 0!)) \\
&= 3 \times (2 \times 1) \\
&= 3 \times 2 \\
&= 6
\end{aligned}
$$

栈与递归

- **阿克曼函数**
  - W. Ackermann, 1928；
  - 一个‘可计算’的非原始递归函数；
  - 简化版本：$A(m,n)=\begin{cases} n+1 & if\ m=0 \\ A(m-1,1) & if\ m>0\ and\ n=0 \\ A(m-1,A(m,n-1)) & if\ m>0\ and\ n>0 \end{cases}$
  - 递归实现：

```
int A(unsigned int  m, unsigned int n){
      if(m==0) return n+1;
      else{
            if(n==0) return A(m-1,1);
            else  return A(m-1, A(m,n-1));
      }
}
```

A(4,2) ?

$$A(m,n) = \begin{cases} n+1 & if\ m=0 \\ A(m-1,1) & if\ m>0\ and\ n=0 \\ A(m-1,A(m,n-1)) & if\ m>0\ and\ n>0 \end{cases}$$

$$A(0,n) = n+1$$

$$A(0,A(1,n-2))$$

$$A(1,n) = A\big(0,A(1,n-1)\big) = 1 + A(1,n-1) = \cdots = n+2$$

$$A(2,n) = A\big(1,A(2,n-1)\big) = A(2,n-1)+2 = \cdots = 2n+3$$

$$A(3,n) = A\big(2,A(3,n-1)\big) = 2A(3,n-1)+3 = \cdots = 2^{n+3}-3$$

$$A(4,n) = A\big(3,A(4,n-1)\big) = 2^{A(4,n-1)+3}-3 = \cdots = \mathbf{2 \uparrow\uparrow (n+3) - 3}$$

… …

$$\mathbf{2 \uparrow\uparrow (n+3) = 2^{2^{\cdot^{\cdot^{\cdot^{2}}}}} \Big\} (n+3)}$$

$$A(4,0) = 2^{2^2} - 3 = 13$$

$$A(4,1) = 2^{2^{2^2}} - 3 = 65533$$

$$A(4,2) = 2^{65536} - 3 > \mathbf{10^{16384}}$$

… …

- 越界！
  - int $(-2^{31} \sim +2^{31}-1)$
  - double $(-10^{308} \sim +10^{308})$
- 很可能A(4,1)都算不出来！?

http://www.gfredericks.com/sandbox/arith/ackermann

栈与递归

- **斐波那契的兔子**
  - 《Liber Abaci》, Fibonacci,1202
  - 某人买了一对兔子，养殖在封闭的环境内；
  - 每对兔子每个月生一对小兔子；
  - 小兔子两个月大后具备生育能力；
  - 一年之后总共有多少对兔子?

月份		一	二	三	四	五	六	七	八	九	十	十一	十二
对兔	1	1	2	3	5	8	13	21	34	55	89	144	233

$1+1$

$2+1$

$3+2$ ……

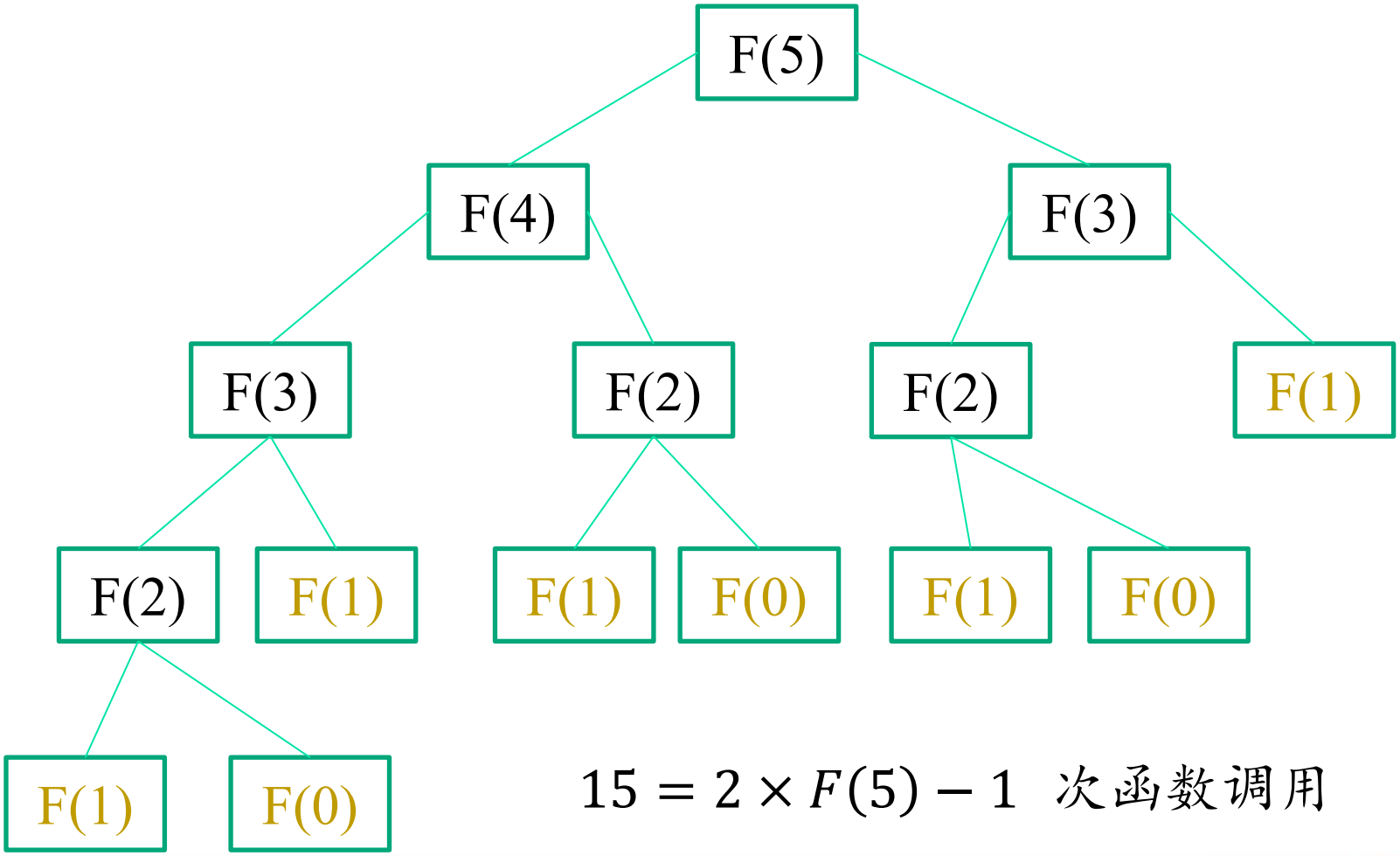
栈与递归

- **斐波那契数列**
  - 递归定义：$F(n)=\begin{cases}1 & n=0,1\\ F(n-1)+F(n-2) & n>1\end{cases}$
  - 数列的通项目：$F(n)=\frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n-\left(\frac{1-\sqrt{5}}{2}\right)^n\right]$
- **递归实现：**

```
long F(unsigned long n){
      if(n<=1) return 1;
      else return F(n-1)+F(n-2);
}
```

栈与递归

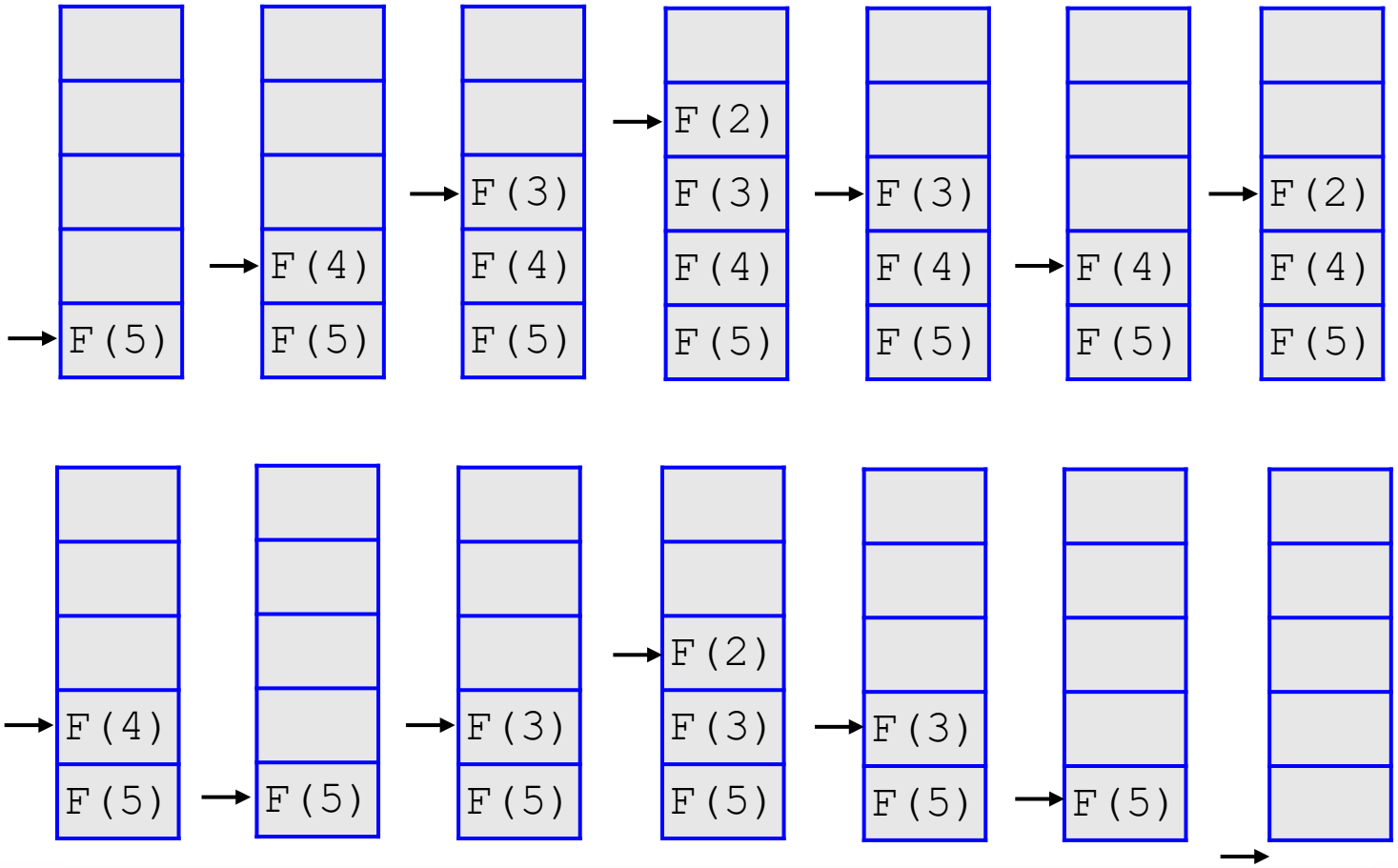
$F(5)$ 的递归调用树



$15 = 2 \times F(5) - 1$ 次函数调用

栈与递归

$F(5)$ 的调用栈情况



递归

- 基本思想：问题经过变换（通常是规模缩减，更接近停止条件）仍然可以用相同的方法解决
- 回文字符串：首尾字符相同，中间子串也是回文的
- N个元素的全排列：第1个位置逐个取1~N号元素，第2-N个位置是剩余N-1个元素的全排列。（N个元素的子集类似）
- （1）问题转换为相同问题；（2）停止条件
- ~数学归纳法 Recursive leap of faith

递归

- **递归方法的优点：**
  - 算法思路清晰，常可以直接从问题定义出发；
  - 程序代码简洁，易读性强，且容易验证；
  - 一种重要的编程工具，开发原型算法用于测试.
- **递归算法的问题：**
  - 大量的函数调用（主要是切换上下文）导致程序运行效率低下；
  - 多层函数调用消耗大量的栈空间，容易产生溢出.
- **递归的消去：**
  - 可以证明递归与循环控制结构的表达能力等价；
  - 用一组非递归语句代替递归调用称为递归消去.

递归

- **递归的消去：**
  - 提高程序空间和时间效率，使程序对堆栈空间的使用得到控制
  - 原则上一定可以通过设置显性栈代替递归调用栈的方式消去递归
  - 某些递归方式，如尾递归，可以直接用循环的方式加以消除；迭代方式也可以消除某些递归
  - 尾递归：函数（每个分支）最后操作是递归调用(或平凡返回)
    - Caller中的上下文不再使用
    - Callee返回时的下一步运行逻辑是Caller的返回操作
    - 思考调用栈的变化：尾递归→循环

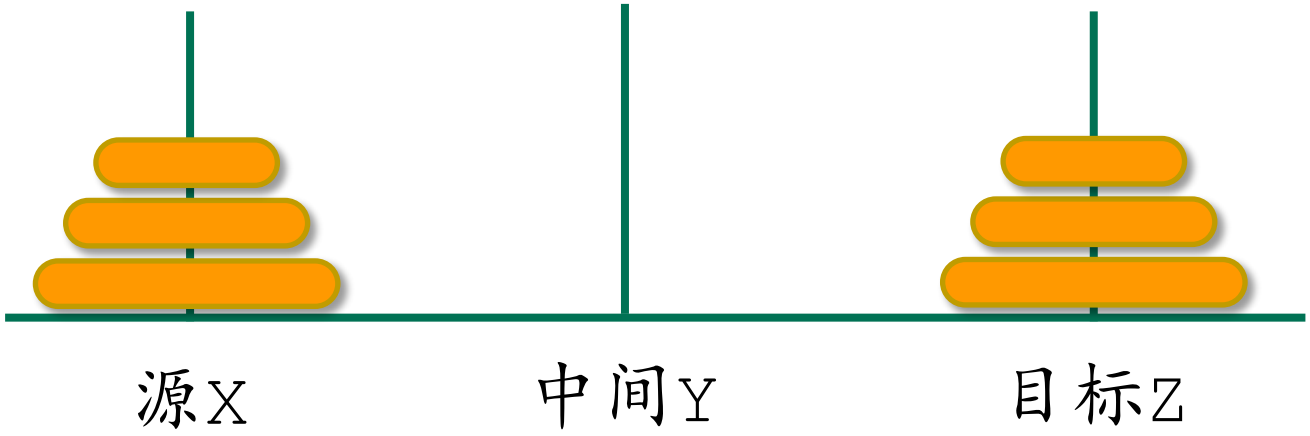
栈与递归

- **最大公约数**
  - 可以同时除尽两个正整数的最大的整数；
  - 辗转相除法(《Elements》 Euclid, 300BC)
    - 求a,b的公约数 $\gcd(a,b)$
    - $\gcd(a,0)=a, \gcd(a,b)=\gcd(b, a\ mod\ b)$

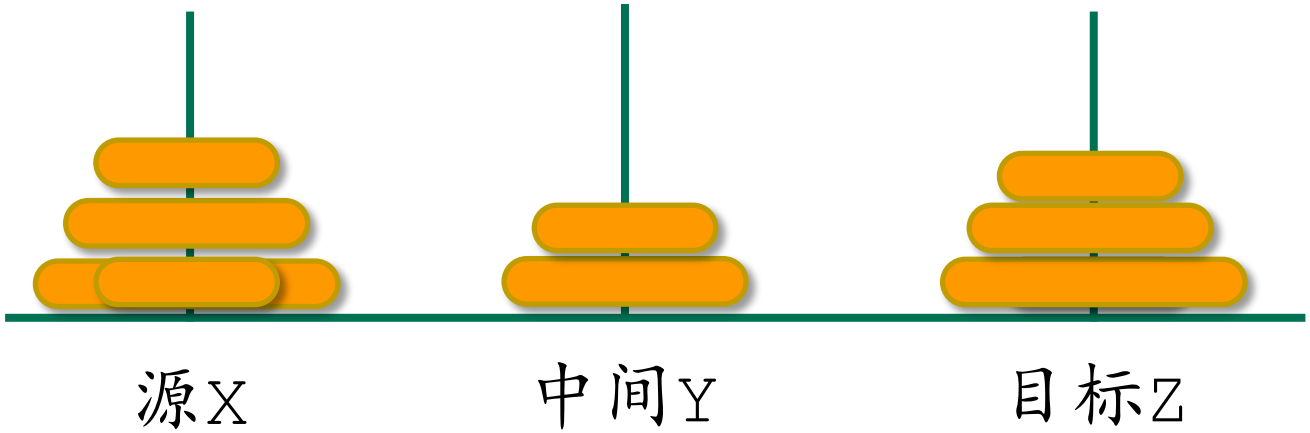
```
long gcd(unsigned long a, unsigned long b){
      int r = a%b;
      if(r!=0) return gcd(b, r);
      else return b;
}
```

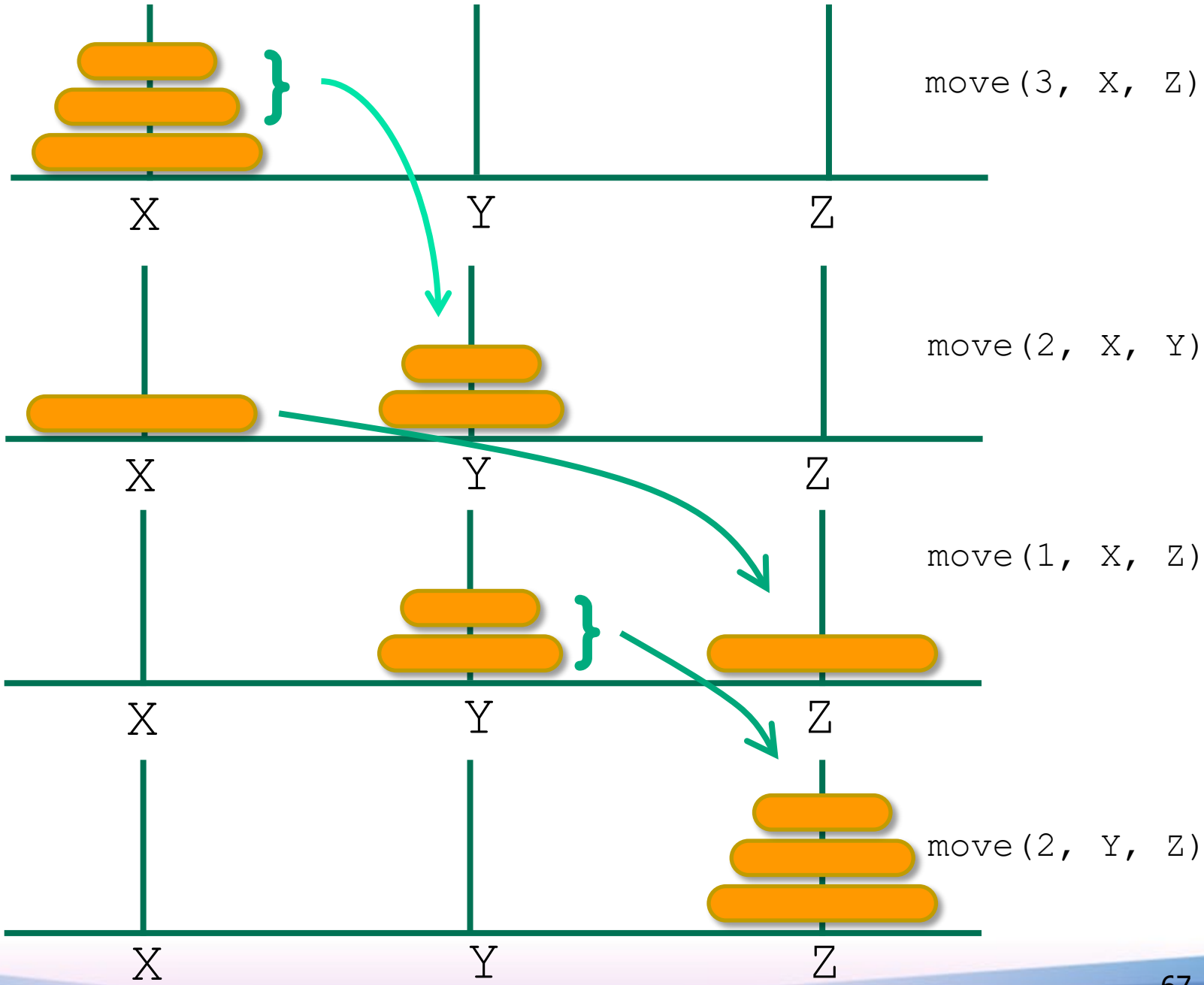
栈与递归

- **汉诺塔问题**
  - 最早由法国科学家E.A. Lucas于1883年提出；
  - 一个古老的印度神庙里有三根柱，其中一个自上而下放置了由小到大的64个金盘，僧侣们依照以下规则把64个金盘移动到另一根柱子上：
    - 一次只能移动一个金盘；
    - 金盘只能在柱子上存放；
    - 小盘必须始终放置在大盘上方；
  - 传说64个金盘移动完世界毁灭 ...



- 将‘源X’柱上的金盘依规则搬运到‘目标Z’柱上，‘中间Y’柱可以用来暂存金盘；
- 没有明显的递归定义，但可转化为递归问题.


源X
中间Y
目标Z


move(3, X, Z)
X
Y
Z
move(2, X, Y)
X
Y
Z
move(1, X, Z)
X
Y
Z
move(2, Y, Z)
X
Y
Z

栈与递归

- **定义函数move()，将金盘1,2,...,n从x移动到z，以y为暂存:**

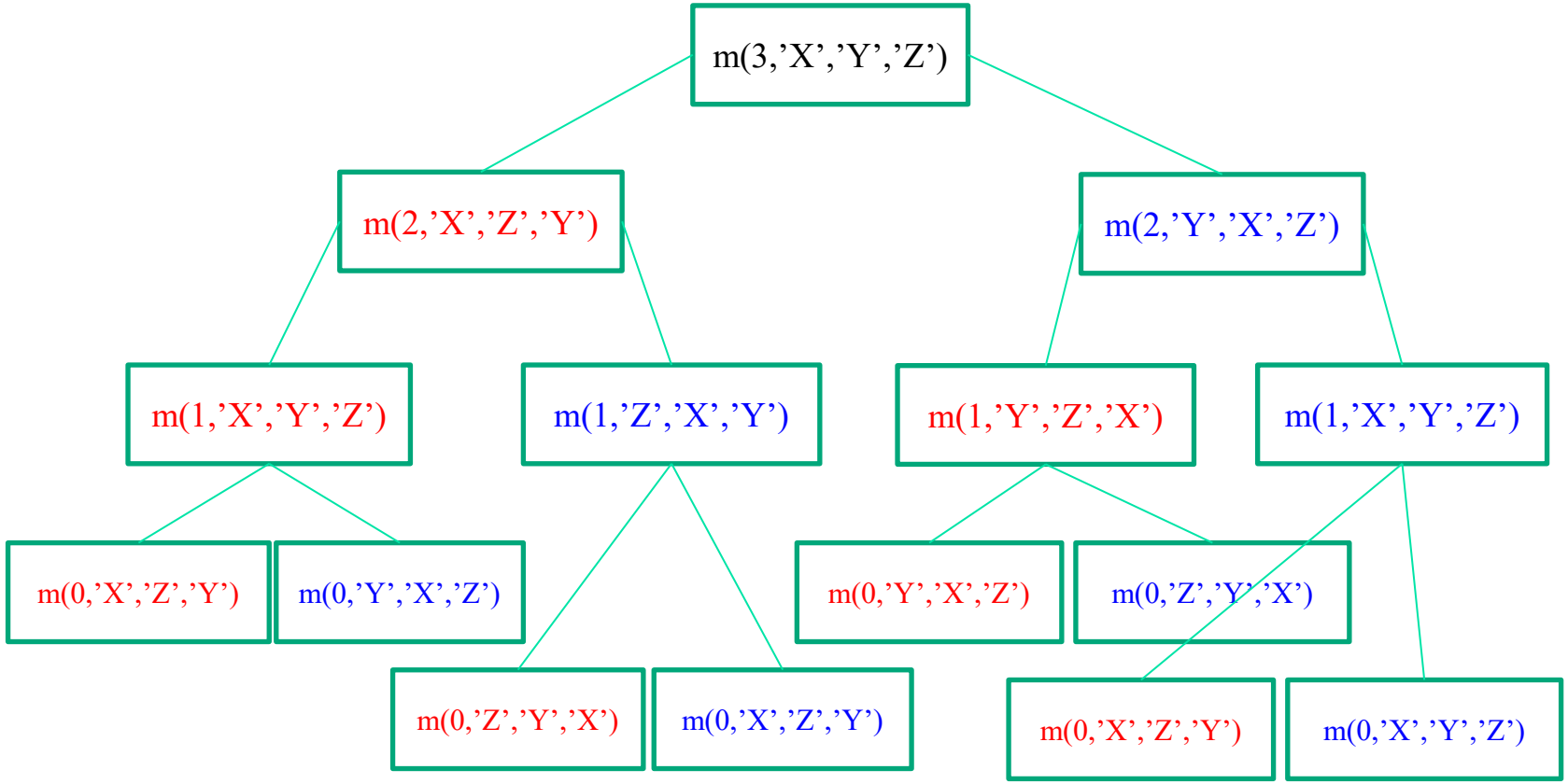
```
void move(int n, char x, char y, char z){
 if (n>0){
    move(n-1, x, z, y);
    cout<<"Move disk "<<n
        <<" from "<<x<<" to "<<z<<endl;
    move(n-1, y, x, z);
 }
}

  move(3,'X', 'Y', 'Z');
```

运行结果:

```
Move disk 1 from X to Z
Move disk 2 from X to Y
Move disk 1 from Z to Y
Move disk 3 from X to Z
Move disk 1 from Y to X
Move disk 2 from Y to Z
Move disk 1 from X to Z
```

栈与递归

move(3,'X', 'Y', 'Z') 的递归调用树



move(64,'X', 'Y', 'Z')?

栈与递归

- 设有n个金盘的Hanoi的移动次数为 M(n)：
- $M(n) = \begin{cases} 0 & n = 0 \\ 2M(n-1) + 1 & n > 0 \end{cases}$
- 令 $t(n) = M(n) + 1 = \begin{cases} 1 & n = 0 \\ 2t(n-1) & n > 0 \end{cases}$
- 可得 $t(n) = 2^n$，或 $M(n) = 2^n$-1；
- 64个金盘需移动约1.8×10^{19}次；
- 僧侣一秒钟移动一个需超过5000亿年，而宇宙的至今不过137亿年！
- 电脑函数调用约需8万年…

栈与递归

- **递归方法的优点：**
  - 算法思路清晰，常可以直接从问题定义出发；
  - 程序代码简洁，易读性强，且容易验证；
  - 一种重要的编程工具，开发原型算法用于测试.
- **递归算法的问题：**
  - 大量的函数调用导致程序运行效率低下；
  - 多层函数调用消耗大量的栈空间，容易产生溢出.
- **递归的消去：**
  - 可以证明递归与循环控制结构的表达能力等价；
  - 用一组非递归语句代替递归调用称为递归消去.

单选题 1分

设置

下述阶乘函数是尾递归么？

```
int factorial(int  n){
    if(n==0) return 1;
    else return n*factorial(n-1);
}
```

A 是

B 否

提交

栈与递归

- **递归的消去：**
  - 提高程序空间和时间效率，使程序对堆栈空间的使用得到控制
  - 原则上一定可以通过设置显性栈代替递归调用栈的方式消去递归；
  - 某些递归方式，如尾递归，可以直接用循环的方式加以消除；迭代方式也可以消除某些递归.

尾递归构造思想

```
int factorial(int  n){
    if(n==0) return 1;
    else return n*factorial(n-1);
}
```

1

不属于尾递归，但是很容易改写为尾递归。现代编译器甚至可以自动将其改为尾递归并优化

```
int factorial(int  n){
    int acc = 1;
    while(n>0){
        acc = acc*(n--);
    }
    return acc;
}
```

尾递归构造思想

$$n!\,C = (n-1)!\cdot Cn$$

1 *尾递归转换通常需要引入额外的参数（升维），用于保存状态*

```
int factorialT(int  n, int C=1){
    if(n==0) return acc;
    else return factorialT(n-1, C*n);
}
```

```
int factorialT(int  n, int C=1){
    while(1){
        if(n==0) return C;
        C = C * n;
        n = n-1;
    }
}
```

尾递归构造思想

- **思想：尾递归与迭代**
  - 尾递归 $f(X1) = f(X2) = f(X3) = \cdots$ 此处Xi是参数构成的向量
  - 本质：$X1$? $X2 \rightarrow X3$ ... 直到递归结束（条件满足）
  - 改写为尾递归的方法，在更高的维度上找出等式

$$n! = (n-1)! * n$$

$$n! * C = (n-1)! * (n * C)$$

  - 尾递归优化: 在循环中直接修改函数输入

尾递归实现和消除

- **斐波那契数列**
  - 尾递归实现 $F(n) = F(n-1) + F(n-2)$ 需要找到不变量

$$
\begin{aligned}
& f(n) + a \cdot f(n-1) + b \cdot f(n-2) \\
& = (1+a)f(n-1) + (1+b)f(n-2) \\
& = f(n-1) + a(f(n-2) + f(n-3)) + (1+b)f(n-2) \\
& = f(n-1) + (1+a+b)f(n-2) + a \cdot f(n-3)
\end{aligned}
$$

  - 在升2维之后定义函数： $G(n,a,b) = f(n) + a \cdot f(n-1) + b \cdot f(n-2)$
  - 则存在 $G(n,a,b) = G(n-1, a+b+1, a)$

    $G(0,a,b) = 1 + b$
  - 故： $f(n) = G(n,0,0)$

1 *注意：高维上G(n)只依赖于G(n-1)*

尾递归实现和消除

- **斐波那契数列**
  - 尾递归实现

$$G(n,a,b) = f(n) + a \cdot f(n-1) + b \cdot f(n-2)$$

```
int G(int n, int a, int b){
  if(n==0) return 1+b;
  else return Gi(n-1, 1+a+b, a);
}
```

  - 尾递归消除：

```
int Gi(int n, int a, int b){
  while(1){
    if (n==0) return 1+b;
    int temp = 1+a+b;
    b = a;
    a = temp;
    n = n-1;
  }
}
```

1 *注意：更新上下文*

尾递归实现和消除

- **斐波那契数列**
  - Bottom-Up的迭代算法

```
int F(int n){
  if(n<=1) return 1;
  int a, b, r, m;
  a = b = 1;
  m = 1;
  do {
      r = a + b;
      b = a;
      a = r;
      m ++;
  } while(m < n );
  return r;
}
```

```
boolean DFS(Node cur, Node target, Set<Node> visited) {
    return true if cur is target;
    for (next : each neighbor of cur) {
        if (next is not in visited) {
            add next to visted;
            return true if DFS(next, target, visited) == true;
        }
    }
    return false;
}
/*
 * Return true if there is a path from cur to target.
 */
boolean DFS(int root, int target) {
    Set<Node> visited;
    Stack<Node> s;
    add root to s;
    while (s is not empty) {
        Node cur = the top element in s;
        return true if cur is target;
        for (Node next : the neighbors of cur) {
            if (next is not in visited) {
                add next to s;
                add next to visited;
            }
        }
        remove cur from s;
    }
    return false;
```

隐性栈

显性栈

栈与递归

- **阶乘求值**
  - 递归实现

```
int factorial(int  n){
    if(n==0) return 1;
    else return n*factorial(n-1);
}
```

  - 循环实现

```
int factorial(int  n){
    int acc = 1;
    while(n>0){
        acc = acc*(n--);
    }
    return acc;
}
```

栈与递归

- **阶乘求值**
  - 尾递归：函数返回前最后一个操作是递归调用
  - 多数现有的编译器可以自动将尾递归优化为循环；
  - 尾递归实现

```
int factorialT(int  n, int acc){
    if(n==0) return acc;
    else return factorialT(n-1, acc*n);
}
```

  - 尾递归优化: 在循环中直接修改函数输入.

```
int factorialT(int  n, int acc){
    while(1){
        if(n==0) return acc;
        acc = acc * n;
        n = n-1;
    }
}
```

栈与递归

- **斐波那契数列**
  - 尾递归实现

```
int FT(int n, int acc1, int acc2){
      if(n<1) return acc1;
      else return FT(n-1, acc2, acc1+acc2);
}
```

  - 尾递归优化

```
int FT(int n, int acc1, int acc2){
      while(1){
          if(n<1) return acc1;
          acc2 = acc1+acc2;
          acc1 = acc2;
          n = n-1;     }
}
```

栈与递归

- **斐波那契数列**
  - 尾递归实现

```
int FT(int n, int acc1, int acc2){
        if(n<1) return acc1;
        else return FT(n-1, acc2, acc1+acc2);
}
```

  - 尾递归优化

```
int FT(int n, int acc1, int acc2){
      while(1){
          if(n<1) return acc1;
           int tmp1 = acc1, tmp2 = acc2;
           acc2 = tmp1+tmp2;
           acc1 = tmp2;
           n = n-1;     }
}
```

栈与递归

- **斐波那契数列**
  - 迭代算法

```
int F_iter(int n){
    if(n<=1) return 1;
    int twoback=1, oneback=1, current;
    for (int i=2;i<=n;i++){
        current = oneback+twoback;
        twoback = oneback;
        oneback = current;
    }
    return current;
}
```

栈与递归

- **斐波那契数列**
  - 设递归实现的函数调用次数 $C(n)$

```
long F(unsigned long n){
        if(n<=1) return 1;
        else return F(n-1)+F(n-2);
}
```

  - $C(n) = C(n-1) + C(n-2) + 1 \;=>\; C(n) = 2F(n) - 1$
  - $F(n) = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]$
  - 递归算法的时间复杂度为 $O(t^n)$
  - 尾递归(优化)、迭代算法的时间复杂度均为$O(n)$.

栈与递归

不同算法的耗时(秒)

n	20	30	40	50	60
F(n)	10946	1346269	165580141	2.0365e+10	2.5047e+12
递归	<0.01	0.01	1.31	194.74	>7200
尾递归	<0.01	<0.01	<0.01	<0.01	<0.01
迭代	<0.01	<0.01	<0.01	<0.01	<0.01

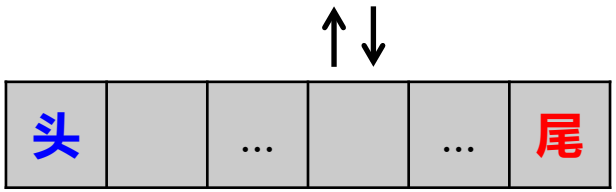
本节内容

- 基本概念
- 栈与递归
- 队列

基本概念

- **栈、队列、双端队列均是线性表的特殊情况（访问受限的线性表）**

一般线性表
'随进随出(RIRO)'



Insert()
Delete()

栈
'后进先出(LIFO)'

Push()
Pop()

队列
'先进先出(FIFO)'

EnQueue()
DeQueue()

双端队列
'先后进出?'

EnQueue_Front()
Dequeue_Front()
EnQueue_Back()
DeQueue_Back()

队列

- 队列：限制数据一端插入，另一端删除的特殊线性表；
- 允许插入的一端称为队尾(rear)，允许删除的一端称为队头(front)；
- 插入元素为入队(Enqueue)，删除元素为出队(DeQueue)；
- 队中元素的个数称为队的长度；

队列

- **队列的ADT**

```
ADT Queue{
    数据对象: D={a_i | a_i∈ElemSet, i=0,1,…,n-1}
    数据关系: R={<a_i, a_i+1> | a_i∈D, i=0,1,…,n-2}
              a_0为队头, a_n-1为对尾.
    基本操作:

    InitQueue(&Q)
        操作结果: 构造一个空队列.

    DestroyQueue(&Q)
        初始条件: 队列Q已存在.
        操作结果: 销毁队列Q.
```

队列

……

QueueEmpty(Q)

初始条件：队列Q已存在.

操作结果：若Q空，返回1，否则返回0.

QueueLength(Q)

初始条件：队列Q已存在.

操作结果：返回Q的长度n.

GetFront(Q,&e)

初始条件：队列Q已存在且非空.

操作结果：用e返回Q的队头元素值.

……

队列

……

EnQueue(&Q,e)

初始条件：队列S已存在.

操作结果：插入e为新的队尾元素.

DeQueue(Q,&e)

初始条件：队列Q已存在且非空.

操作结果：删除队头元素，并用e返回其值.

Clear(&Q)

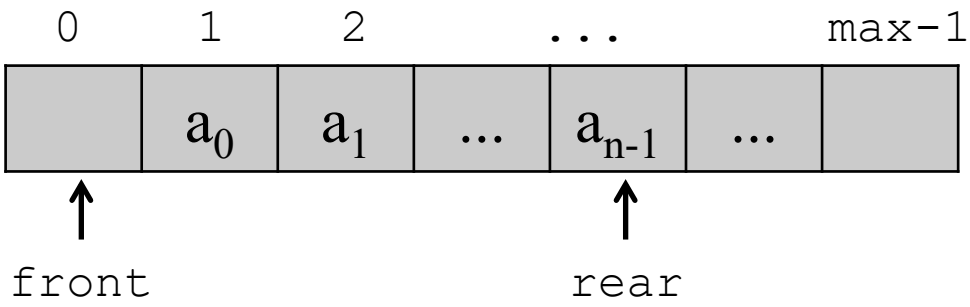
初始条件：队列Q已存在.

操作结果：清空队列Q，清空后队列长度为0.

} ADT Queue

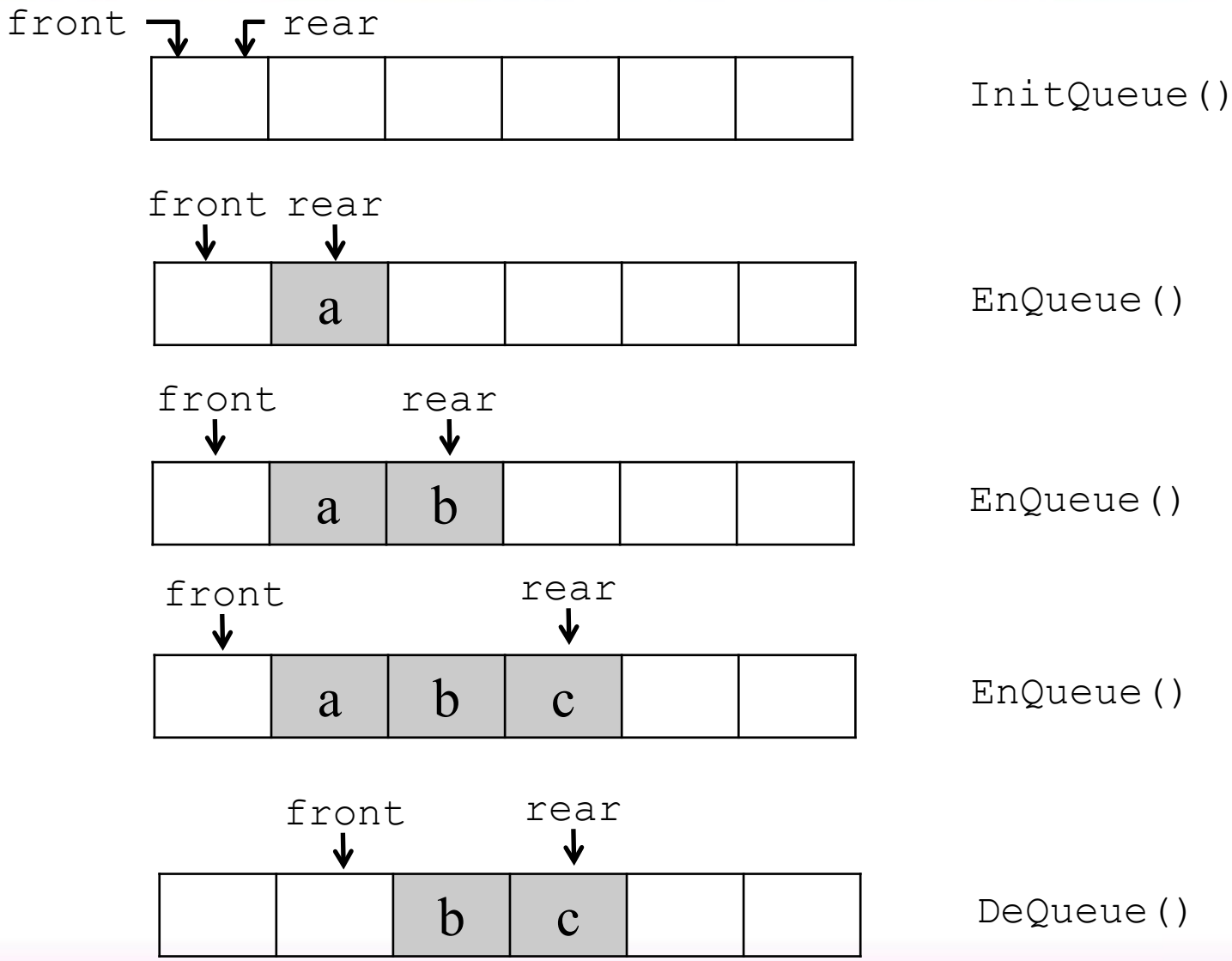
队列

- **顺序队列**
  - 队的顺序存储实现

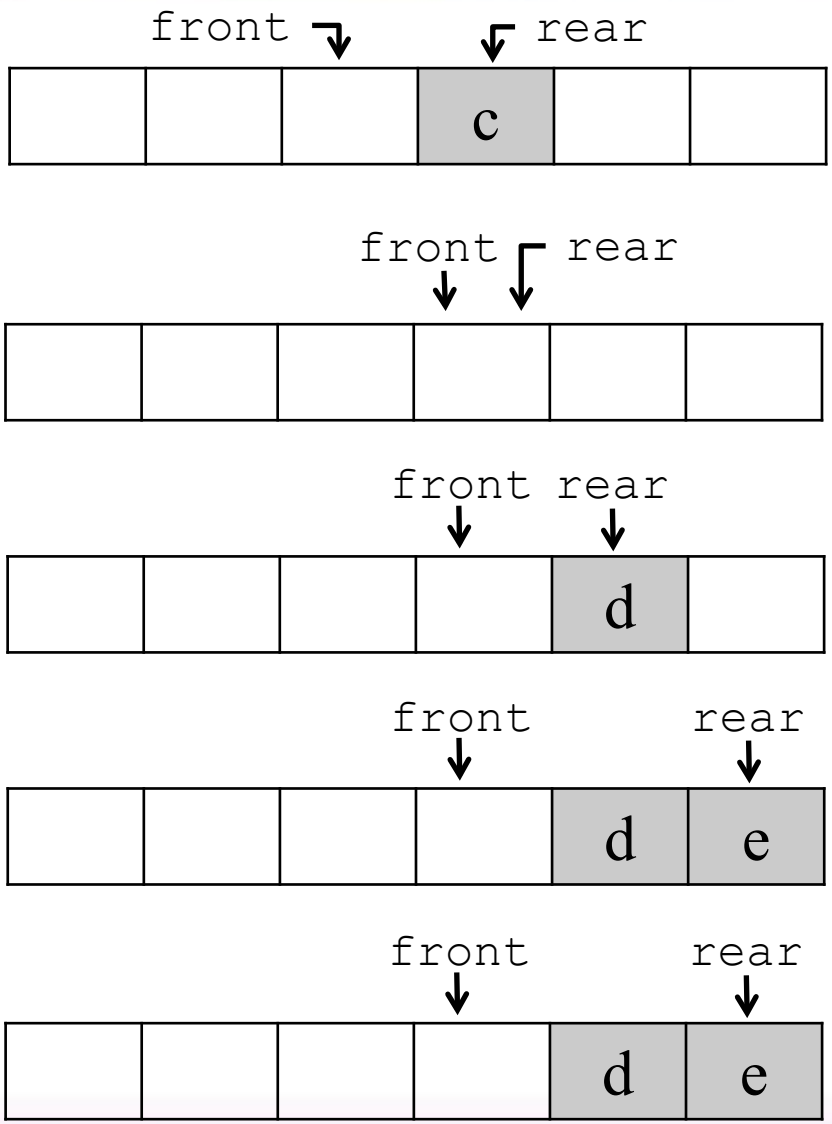



```
template <class ElemType>
class SeqQueue{
        ...
        bool EnQueue(ElemType e)
            { if(rear==max-1) return false;   //溢出
              queue[++rear]=e; return true;}
        bool Dequeue(ElemType &e)
            { if(front==rear) return false;   //队列空
              e = queue[front++]; return true;}
};
```

队列


front
rear
InitQueue()
front rear
a
EnQueue()
front
rear
a
b
EnQueue()
front
rear
a
b
c
EnQueue()
front
rear
b
c
DeQueue()

队列



DeQueue()

DeQueue()

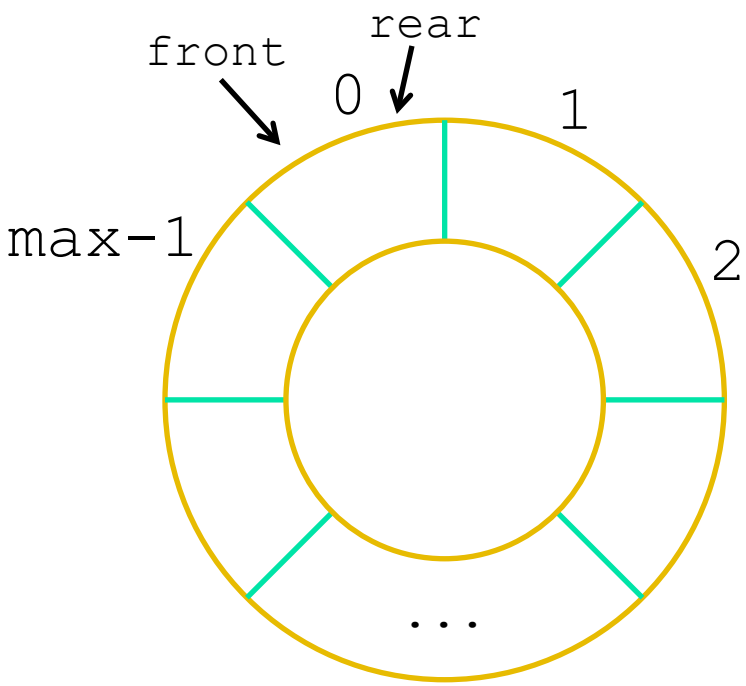
EnQueue()

EnQueue()

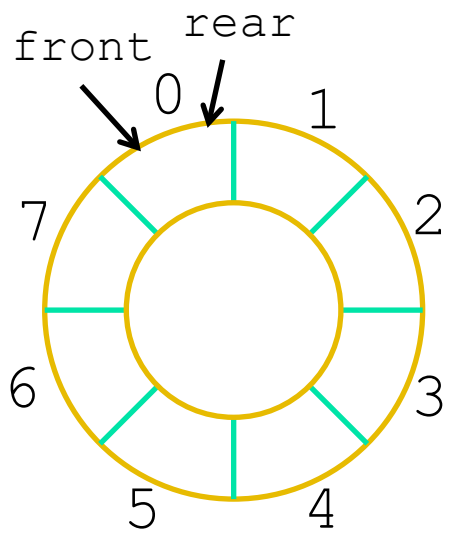
EnQueue() 假溢出！

队列

- **循环队列(circular queue)**
  - 解决顺序队列假溢出问题;
  - 逻辑上使得存储空间首尾相接.

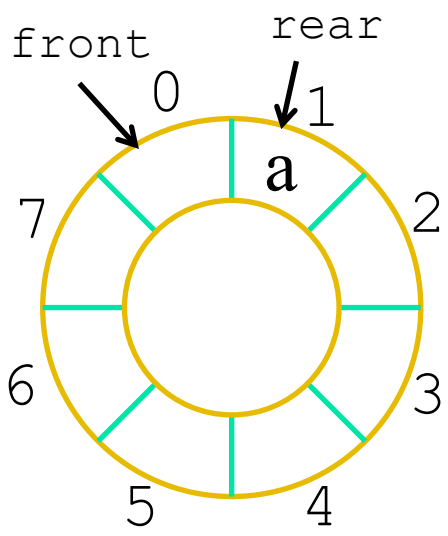



```
EnQueue():
  rear = (rear+1)%max

DeQueue():
  front = (front+1)%max
```

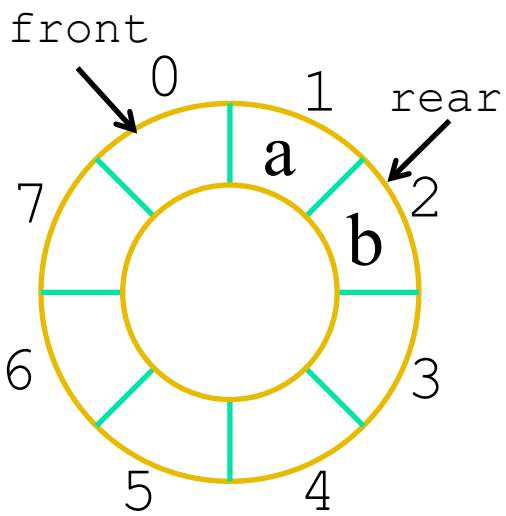


队列空

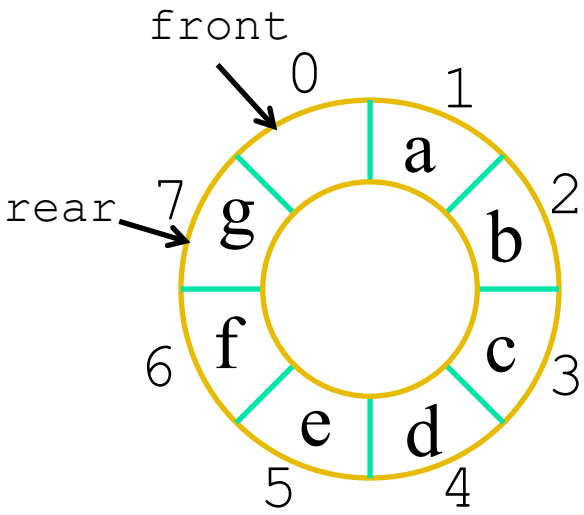
front=rear=0



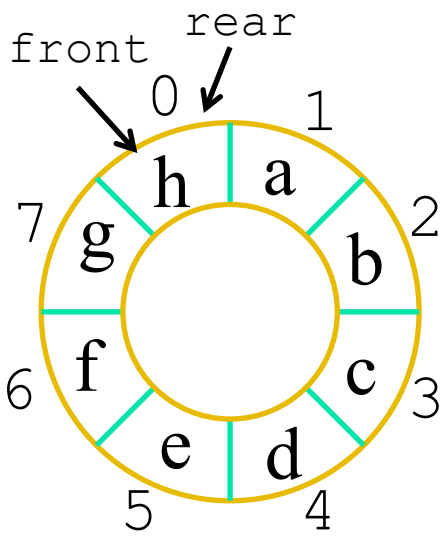
'a' 入队



'b' 入队



'c' ~ 'g' 入队



'h' 入队，队列满

front=rear=0

如何区分

队列空和队列满?

队列

- **循环队列空/满判断**
  - 留出一个空位：
    - 队列为空：front==rear;
    - 队列为满：front==(rear+1)%max;
  - 设置状态标志：
    - 出队操作导致 front==rear，则标志为空；
    - 入队操作导致 front==rear，则标志为满；
    - 其他情况标志队列正常；
  - 设置队列长度变量：
    - 入队则增1，出队则减1；
    - 变量为0则队空，变量等于max则队满.

队列 循环队列的实现

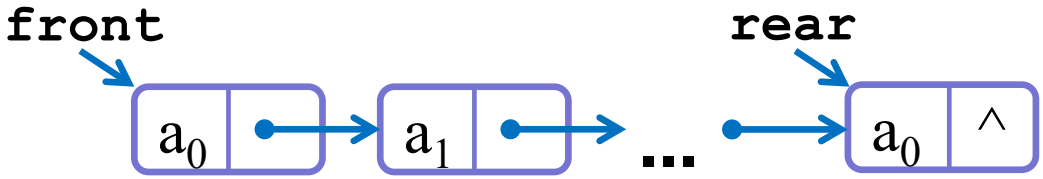
```
template <class ElemType>
class CirQueue{
  private:
    ElemType* queue; int MaxSize, front, rear, len;
  public:
    CirQueue(unsinged int m)
      {MaxSize=m; queue=new ElemType[m]; front=rear=len=0;}
    ~CirQueue() { delete[] queue; }
    bool QueueEmpty() { return len==0; }
    bool QueueFull()  { return len==MaxSize; }
    int QueueLength() { return len; }
    bool EnQueue(ElemType e)
    bool DeQueue(ElemType &e)
};
```

队列

```
template <class ElemType>
bool CirQueue<ElemType>::EnQueue(ElemType e){
    if(QueueFull()) return false;     //队列溢出
    rear = (rear+1)%MaxSize;
    queue[rear] = e;  len++;
    return true;
}

template <class ElemType>
bool CirQueue<ElemType>::DeQueue(ElemType &e){
    if(QueueEmpty()) return false;    //队列空
    e = queue[front]; len--;
    front = (front+1)%MaxSize;
    return true;
}
```

队列

- **链式队列-队列的链式存储分配实现**
  - 用链表的方式存储队列



  - 两个指针分别指向队头和队尾
  - 入队时不存在溢出的问题，能更好的利用空间

结点定义

```
template <class ElemType>
class Node{
    public:
        ElemType data;
        Node<ElemType> * next;
        Node(ElemType e, Node* t)
                {data=e; next=t;}
}
```

队列 链式队列的C++实现

```
template <class ElemType>
class LkQueue{

  private:

    Node<ElemType> *front, *rear; int len;

  public:

    LkQueue()  {front=rear=NULL; len=0;}

    ~LkQueue() { ClearQueue(); }

    void ClearQueue();

    bool QueueEmpty() { return len==0; }

    int QueueLength() { return len; }

    bool EnQueue(ElemType e);

    bool DeQueue(ElemType &e);

};
```

队列

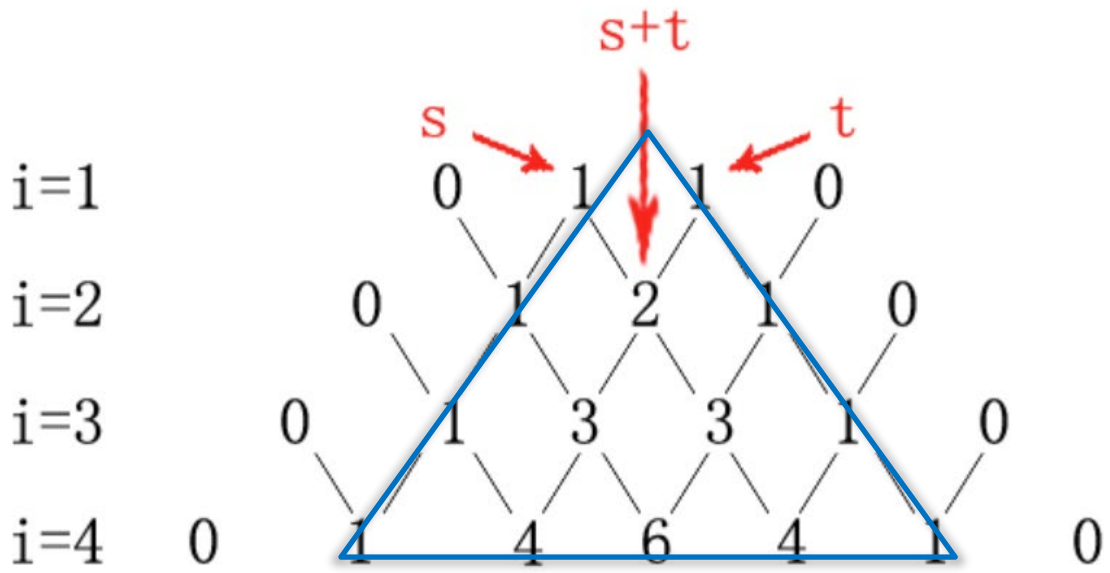
```
template <class ElemType>
void LkQueue<ElemType>::ClearQueue(){
    ElemType e; while(DeQueue(e));
}

template <class ElemType>
bool LkQueue<ElemType>::EnQueue(ElemType e){
    Node<ElemType>* s=new Node<ElemType>(e, NULL);
    if(rear==NULL) {rear=s; front=s;}
    else {rear->next=s; rear=s;}
    len++; return true;
}

template <class ElemType>
bool LkQueue<ElemType>::DeQueue(ElemType &e){
    if(front==NULL) return false;
    e=front->data;
    Node<ElemType>* q=front; front=front->next;
    delete q; len--;
    if(len==0) {front=NULL; rear=NULL;}
    return true;
}
```

队列

- **优先队列 (priority queue)**
  - 队列中每个元素带有一个优先权值；
  - 入队与普通队列相同；
  - 优先级高的元素始终先出队；
  - 出队时可遍历队列找出最高优先级元素；
  - 用线性表实现则出队的算法复杂性较高O(n).

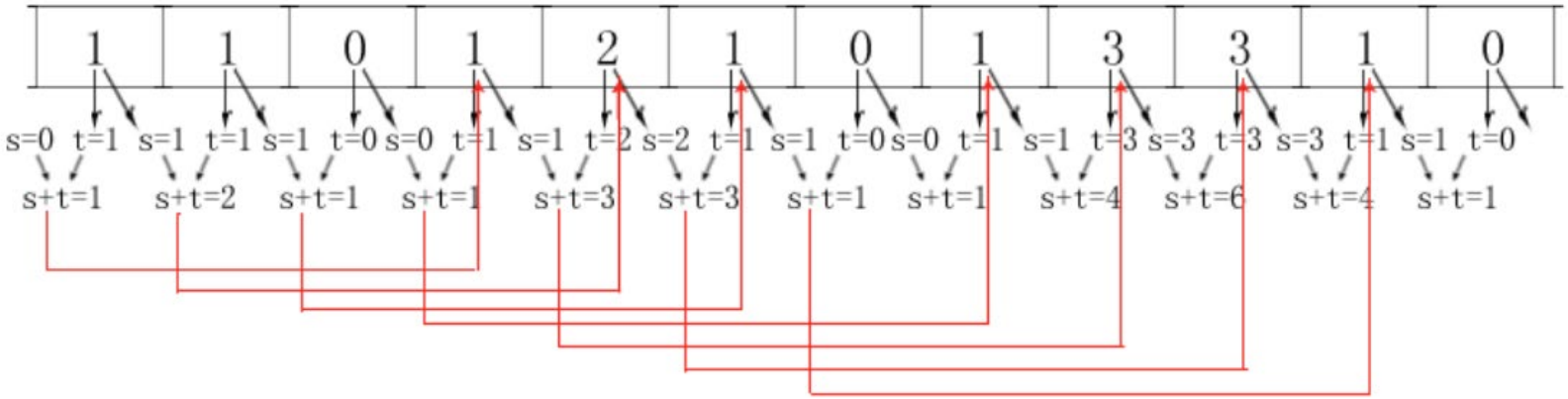
队列

- **队列的应用性**
  - 队列常用于需按照顺序依次处理的问题:
    - 路由器采用队列暂存网络包用于转发;
    - 操作系统使用消息队列用于进程间或者线程间通信;
    - 排队论(queuing theory)：研究排队现象随机规律;
  - 用于缓冲技术，以匹配不同速度的设备:
    - 计算机向慢速的外设设备传送数据，如打印机;
    - 信息的采集速度快于处理速度，如音频、视频的暂存.
  - 其他编程应用
    - 如表达式线性化，Pascal三角形（杨辉三角形）计算.



第i行元素与第i+1行元素的关系

- **从第i行的数据出队并计算出第i+1行的数据入队的示意图：**



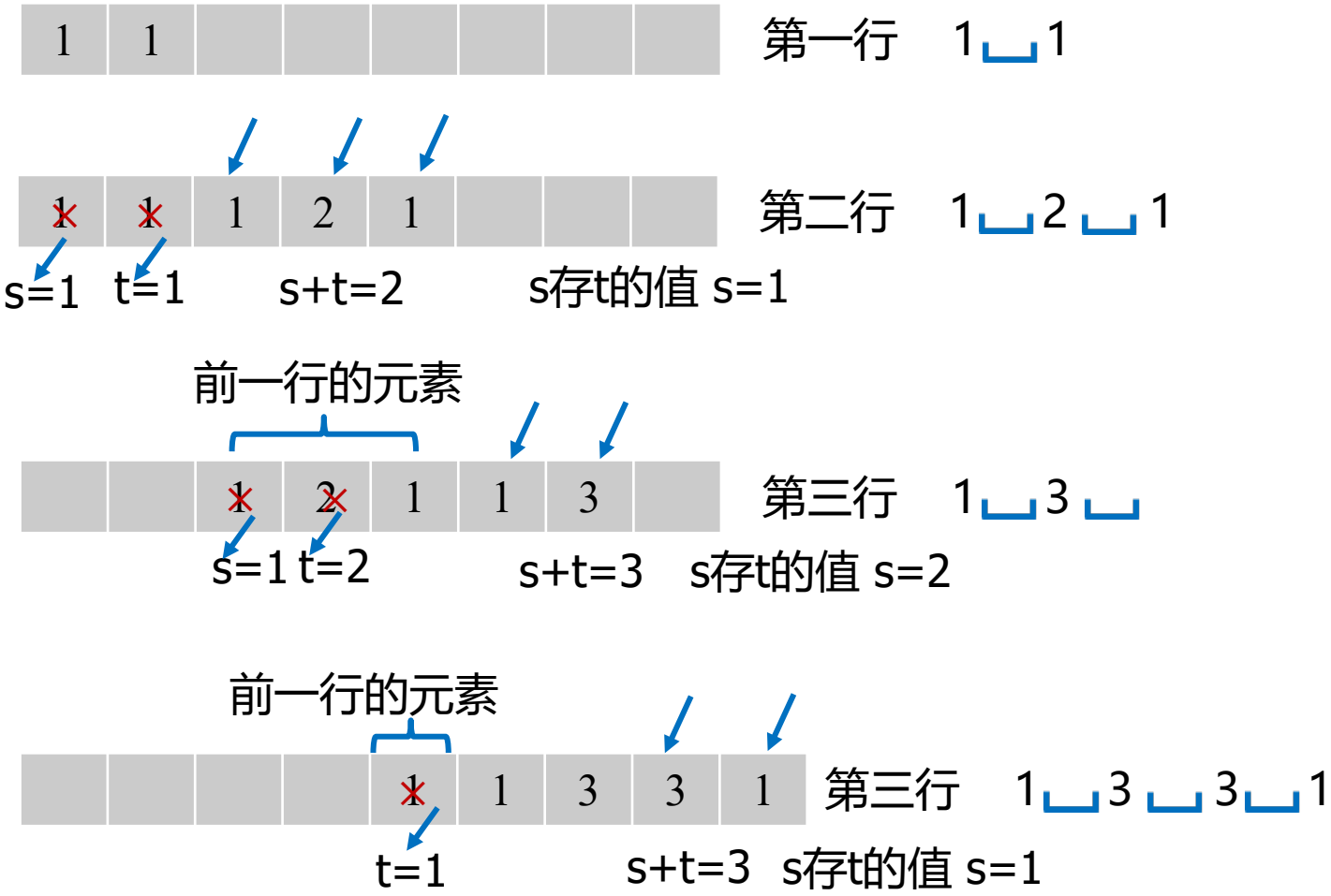
队列

$n=1$ 1 1
$n=2$ 1 2 1
$n=3$ 1 3 3 1
$n=4$ 1 4 6 4 1
… ……

- **Pascal三角形计算：二项式$(a+b)^n$展开后的系数**

```
void PascalTri(int n){
    LkQueue<int> Q;  int s, t;
    Q.EnQueue(1); Q.EnQueue(1);          //第一行 1 1
    std::cout<<1<<' '<<1;
    for(int i=2;i<=n;i++){
        std::cout<<std::endl;
        Q.EnQueue(1); std::cout<<1<<' '; //每行头为1
        Q.DeQueue(s);
        for(int j=2;j<=i;j++){
            Q.DeQueue(t); Q.EnQueue(s+t);
            std::cout<<s+t<<' '; s=t;
        }
        Q.EnQueue(1); std::cout<<1;      //每行尾为1
    }
    std::cout<<std::endl;
}
```

队列


1 1
第一行 1 1
1 1 1 2 1
第二行 1 2 1
s=1 t=1 s+t=2 s存t的值 s=1
前一行的元素
1 2 1 1 3
第三行 1 3
s=1 t=2 s+t=3 s存t的值 s=2
前一行的元素
1 1 3 3 1
第三行 1 3 3 1
t=1 s+t=3 s存t的值 s=1

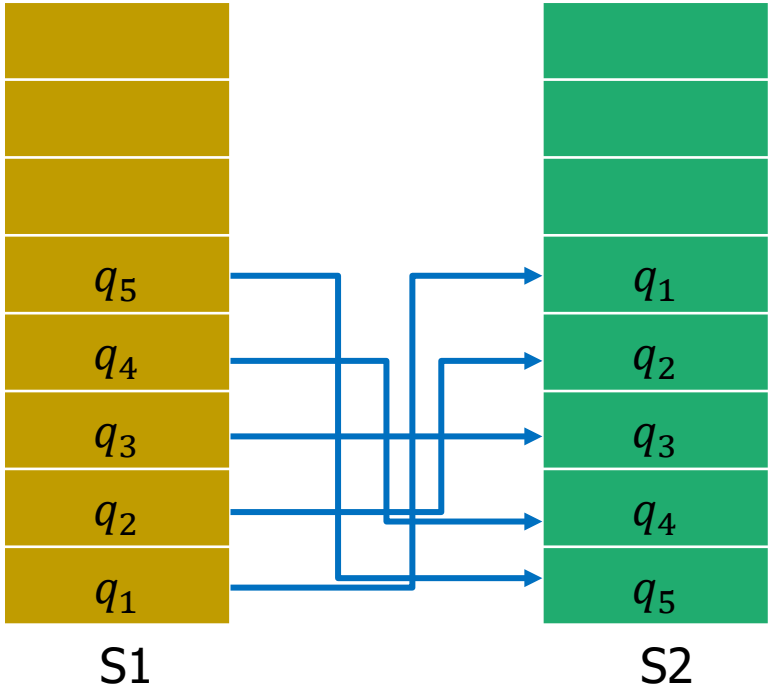
队列 利用两个栈的对象实现一个队列?

```
Stack<int> s1, s2;
...
void EnQueue(int e){
      s1.push(e);
}
bool DeQueue(int &e){
      if (!s2.empty()){
            e = s2.pop(); return true;
      }
      if (s1.empty()) return false;
      while (!s1.empty()) s2.push(s1.pop());
      e = s2.pop(); return true;
}
```

队列 利用两个栈的对象实现一个队列?

q_1	q_2	q_3	q_4	q_5					

Delete 从 q_1 开始




```
s2.push(s1.pop())
```

谢　谢！

北京科技大学
计算机与通信工程学院
杜　冰